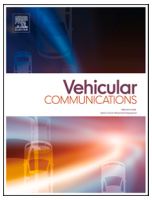
Vehicular Communications 58 (2026) 101005

Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom



A gated transformer MADDPG algorithm for latency and energy aware task offloading in digital twinning Aerial edge computing

Md. Abdullah Al Sami [a], Ibrahim Tanvir [a], Palash Roy [b,*], Md. Abdur Razzaque [b,*], Md Rafiul Hassan [c], Mohammad Mehedi Hassan [d]

[a] Department of Computer Science and Engineering, Green University of Bangladesh, 1461, Narayanganj Dhaka, Bangladesh
[b] Green Networking Research Group, Department of Computer Science and Engineering, University of Dhaka, Dhaka - 1000, Bangladesh
[c] Department of Computer Science, Central Connecticut State University, New Britain, CT, USA
[d] Information Systems Department, College of Computer and Information Sciences, King Saud University, 11543, Riyadh, Saudi Arabia

ARTICLE INFO

Keywords:
Aerial edge computing (AEC)
Energy consumption
Service latency
Load balancing
Task prioritization
Gated transformer-XL (GTrXL)
Multi-agent deep deterministic policy gradient (MADDPG)

ABSTRACT

Unmanned aerial vehicles (UAVs) have seen breakthroughs in forming Aerial Edge Computing (AEC), which executes computationally intensive tasks generated by Internet of Things (IoT) devices, thanks to their ease of deployment, especially in scenarios where traditional terrestrial base stations are damaged and unable to process tasks due to natural disasters. However, an AEC faces significant challenges due to the limited battery capacity of UAVs and the need for efficient collaboration among them to execute tasks. Existing studies often overlook fine-grained task prioritization and balanced load distribution across UAVs, leading to inefficiencies in energy usage and service delay. In this paper, we have developed an optimization framework for efficiently offloading computationally intensive IoT tasks in a three-stage Digital Twin-enabled multi-UAV-based AEC network environment, which jointly minimizes service latency and energy consumption while ensuring the expected load distribution among the UAVs. The formulated framework is a Mixed-Integer Nonlinear Programming (MINLP) problem, which is inherently NP-hard. To address this, we design GLEMATO, a scalable GTrXL-assisted MADDPG framework that learns high-quality offloading policies through memory-aware task prioritization and cooperative multi-agent decision-making in dynamic AEC scenarios. In GLEMATO, while the GTrXL model ensures adaptive task prioritization by considering factors such as task generation time, energy budget, and application deadlines, while the MADDPG enables decentralized policy learning through sharing cooperative state-actions among UAVs. The experimental results, carried out on the OpenAI Gym simulator platform, demonstrate that the developed GLEMATO framework reduces average energy consumption and service latency by 21.8% and 23.3%, respectively, and increases the average task completion ratio by up to 20.1% for computationally intensive tasks compared to the state-of-the-art approaches.

1. Introduction

In recent years, Unmanned Aerial Vehicles (UAVs) have received immense attention across civil, government, and defense applications, including environmental surveillance, emergency response, border patrol, and disaster rescue operations [1], [2]. Due to their cost-efficiency and flexible deployment, UAV networks promote real-time, energy-aware applications aligned with the objectives of Industry 5.0, thereby drawing substantial interest from both academic and industrial communities. According to Precedence Research [3], the global UAV market was estimated at USD 31.55 billion in 2023 and is expected to reach at USD 169.7 billion by 2033. In scenarios such as natural calamities or military zones, where ground-based stations might be inoperative, or IoT devices remain disconnected, UAVs can reestablish communication and perform tasks in real-time or offload computation-heavy operations to Aerial Edge Computing (AEC) servers, which utilize UAVs' onboard resources for localized execution, which traditional static edge systems cannot support effectively.

Nevertheless, the AEC paradigm encounters major technical challenges, including ensuring real-time processing under network delays and coping with the limited battery capacity of UAVs [2,4–6]. Additionally, a single UAV often cannot manage compute-intensive tasks like post-disaster assessment or wildfire tracking, due to constraints such as limited CPU power, payload limits, and energy consumption. Hence,

* Corresponding authors.
E-mail addresses: abdullah.al.sami05@gmail.com (Md.A.A. Sami), ibrahimtanvir680@gmail.com (I. Tanvir), palash@du.ac.bd (P. Roy), razzaque@du.ac.bd (Md.A. Razzaque), mdrafiul.hassan@ccsu.edu (M.R. Hassan), mmhassan@Ksu.edu.sa (M.M. Hassan).

https://doi.org/10.1016/j.vehcom.2026.101005
Received 6 October 2025; Received in revised form 11 January 2026; Accepted 19 January 2026
Available online 26 January 2026

collaboration among multiple UAVs through UAV-to-UAV (U2U) communication becomes essential for effective task fulfillment [5]. However, such collaboration introduces challenges in task segmentation, communication scheduling, and load balancing. As a result, efficiently distributing IoT-generated tasks to neighboring UAVs while considering service latency and energy constraints, and simultaneously ensuring task deadlines and balanced resource usage, remains a complex engineering research problem.

To address the challenges of delay-sensitive applications within the AEC paradigm, real-time UAV coordination is required. Integrating a Digital Twin (DT) [7,8] into the UAV-AEC ecosystem provides a high-fidelity virtual counterpart of UAV states, task dynamics, and network conditions. This virtual representation enables predictive monitoring, state-aware resource management, and proactive decision-making under rapidly changing environments. By simulating UAV trajectories, computational loads, and energy consumption prior to real-world execution, the DT facilitates intelligent task offloading, adaptive load balancing, and timely system responses, thereby improving the robustness and efficiency of UAV-assisted AEC operations.

Several prior efforts have sought to address the task offloading problem in AEC infrastructures, which are typically formed by networks of Mobile Edge Computing-enabled UAVs (MEC-UAVs). In Chen et al. [1], the authors proposed a multi-UAV edge cloud architecture that focuses on energy-efficient task offloading under resource constraints. Another work in [2] introduced a THz-band communication model to optimize UAV trajectories and resource usage for offloading. It considered IoT-to-UAV task offloading, it did not support U2U cooperation, thereby limiting its applicability for compute-heavy workloads. In Hevesli et al. [6], a Digital Twin (DT) model was utilized to reflect the real-time condition of IoT nodes and UAVs. They designed the D2TEAGN architecture to estimate network states, enhance reliability, and optimize multi-UAV trajectories. However, D2TEAGN framework did not incorporate U2U communication to achieve load balancing among UAVs. In Tang et al. [9], a multi-UAV framework enabled by DT was proposed for assigning task, incorporating a three-layer structure to support task reassignment. While the approach enabled U2U-based load balancing, its reliance on task decomposition into smaller subtasks increased communication overhead and latency, making it unsuitable for time-sensitive applications. However, most existing studies primarily emphasize UAV energy consumption and service latency, with only limited attention given to load distribution and task prioritization. Although some approaches attempt to address these aspects, their effectiveness is constrained by the absence of U2U communication, lack of efficient task distribution strategies, and insufficient fine-grained prioritization for real-time computationally intensive workloads scheduling. Consequently, the joint optimization of latency-energy trade-offs, intelligent computational load balancing, and task prioritization in DT-enabled AEC systems remains an open research challenge.

In this paper, we have developed an optimization framework exploiting Mixed-Integer Nonlinear Programming (MINLP) for a DT-enabled multi-UAV task offloading system to jointly minimize service latency and energy consumption while maintaining computational load balancing among AEC servers. A preliminary version of this work was presented in Al Sami et al. [10], which investigated the optimal distribution of tasks across IoT devices, AEC servers, and an Airship, focusing on the latency-energy trade-offs in task offloading. However, the optimization-based approach becomes computationally intractable in large-scale and highly dynamic AEC environments, and cannot provide solutions within polynomial time. Therefore, in this paper, we develop GLEMATO, a Gated Transformer-XL (GTrXL) enhanced Multi-Agent Deep Deterministic Policy Gradient (MADDPG) framework that enables scalable, computationally intensive task allocation across multiple AEC servers. The GTrXL-based prioritization captures temporal dependencies and highlights delay-critical tasks, strengthening MADDPG's cooperative learning and improving tasks offloading and execution decisions in complex, large-scale UAV-assisted AEC environments. The main contributions of this work are summarized as follows:

- We have developed an optimization framework, exploiting MINLP for a DT-enabled multi-UAV-assisted task offloading system, which brings a trade-off between minimizing latency and energy consumption for executing in three stages (locally, at the connected AEC server, neighboring server, or Airship), while ensuring effective load balancing in the AEC layer.
- Since the MINLP-based optimization problem is an NP-hard problem, we have developed the MADDPG algorithm with the GTrXL mechanism, namely, GLEMATO, to allocate tasks for large-scale AEC environments in polynomial time. In the GLEMATO framework, the core optimization engine MADDPG algorithm enhances its learning policy through the support of GTrXL task prioritization strategy, allowing it to focus on tasks with earlier deadlines, effective collaboration among UAVs to distribute computational loads, thereby improving optimization efficiency and enabling fine-grained offloading decision-making.
- The performances of the GLEMATO framework have been studied on the OpenAI Gym platform, and the simulation results demonstrate a significant performance improvement in terms of service latency and energy consumption as high as 21.8% and 23.3%, respectively, compared to the state-of-the-art approaches.

The remainder of this paper is organized as follows: Section 2 provides an extensive literature review, Section 3 discusses the detailed system model and assumptions; Sections 4 and 5 present the detailed design and the algorithm for the developed GLEMATO framework. Subsequently, the performance analysis is depicted in Section 6, and finally Section 7 provides a concise summary of the work.

2. Related works

In this section, we review the related work on UAV-based task offloading, the role of Digital Twin (DT) enhancing AEC server task management, and MADRL-based optimization strategies for AEC server coordination in offloading tasks.

2.1. Conventional (Non-DT) task offloading approaches in UAV-MEC

Several studies have been conducted to offload tasks in UAV or AEC Server networks, enhancing computational efficiency and energy management. Zhou et al. [11] proposed three different UAV-enabled MEC network architectures based on the role in the network, such as task generators or MEC servers, or relay nodes that forward data. The study highlighted key implementation challenges such as operation modes partial vs. binary offloading, resource allocation, and security issues in a single-UAV scenario, lacking solutions for multi-UAV coordination. Messous et al. [12] employed a non-cooperative game-theoretic model for offloading computationally intensive tasks in AEC networks, optimizing energy consumption, delay, and communication cost, but assumed static network conditions and lacked mechanisms for dynamic task segmentation or multi-server coordination. Chen et al. [1] proposed a hierarchical edge cloud architecture for multi-UAV systems called RESERVE, which introduced local (small UAVs), edge (large UAVs), and cloud layers and developed a game-theoretical algorithm for resource and task allocation. However, they did not divide tasks into subtasks, leading to potential task failure due to the limited resources of UAVs. Similarly, Zhang et al. [13] address the energy-latency trade-off in UAV-assisted MEC environment by proposing a decentralized game-theoretic approach, namely GTCO, where mobile devices (MDs) can intelligently decide optimal offload decisions to a UAV or ground base station. Although GTCO's decentralized design handles MDs' autonomy, however, it does not address scenarios involving multiple UAVs or complex task segmentation strategies, which are critical for managing computationally intensive workloads.

Table 1
Comparative analysis of GLEMATO and state-of-the-art methods.

System	Latency	Energy	Energy Harvesting	Task Prioritization	Load Balancing	U2U Comm.	Offloading Type	DT Integrationl	Algorithm's Nature
RESERVE [1]	✗	✓	✗	✗	✗	✗	Binary	✗	Game-theoretic
AGSP [14]	✓	✓	✗	✗	✗	✗	Hybrid	✗	Metaheuristic (GA + PSO + SA)
GTCO [13]	✓	✓	✗	✗	✗	✗	Binary	✗	Game-theoretic
DQN [9]	✗	✓	✗	Deadline Re-assign	✓	✓	Binary	✓	DQN-based
ConvLSTM-PER-DQN [24]	✓	✓	✗	✗	✗	✗	Binary	✓	MADRL (ConvLSTM + DQN)
D2TEAGN [6]	✓	✓	✗	✗	✗	✗	Partial	✓	DDPG-based
MARS [21]	✓	✓	✗	✗	✓	✓	Hybrid	✓	MADRL + HAFL
GLEMATO (Proposed)	✓	✓	✓	GTrXL	✓	✓	Partial or Full	✓	MADDPG + GTrXL

Yuan et al. [14] proposed a cost-efficient task offloading scheme for layered UAV-based MEC systems using a hybrid metaheuristic method namely adaptive and genetic simulated annealing-based particle swarm optimization (AGSP), which improves latency and energy efficiency for both static and dynamic task workloads. Wang et al. [15] further addressed real-time task offloading and trajectory optimization in UAV-assisted MEC, aiming to minimize delay and energy consumption under dynamic constraints. However, these studies primarily focus on single-UAV or non-cooperative settings and do not account for load balancing across multiple UAVs, limiting their applicability in large-scale or high-demand environments. They also lack mechanisms for dynamic task prioritization, which is essential in aerial edge computing.

Recent works in mobile edge computing have explored online optimization to improve adaptability under uncertainty. For example, [16] introduced an online learning-based incentive mechanism that supports collaborative task offloading while ensuring truthful participation and stable performance. Similarly, the authors in [17] proposed a distributed offloading scheme in which users iteratively learn equilibrium strategies to jointly optimize delay and energy consumption, demonstrating the potential of game-theoretic learning for decentralized decision-making. Nevertheless, online and game-theoretic approaches face scalability challenges and often yield suboptimal results in highly dynamic, large-scale UAV-assisted AEC environments. These limitations motivate the adoption of advanced deep reinforcement learning methods for more adaptive, scalable, and efficient task offloading, as discussed in the following subsection.

2.2. Digital twin for UAV task management

The concept of DT has been applied to IoT and UAV networks to enable real-time decision-making [18]. Tang et al. [9] proposed a DT-assisted Deep Q-Network (DQN) for dynamic task assignment in multi-UAV systems using a three-layer structure (generation, assignment, execution), with airships pre-training DT models to reduce UAV computational load. While this approach improved task completion by prioritizing earliest-deadline tasks and balancing workloads, dividing tasks into subtasks introduced overhead and latency, making it unsuitable for delay-sensitive applications. Guo et al. [19] developed a DT-assisted framework for intelligent task offloading and resource allocation, combining binary and partial offloading with DT-enabled state monitoring. However, it assumed homogeneous UAV capabilities and did not consider environmental dynamics, limiting its effectiveness in heterogeneous scenarios.

Hazarika et al. [20] introduced RADiT, a DT-driven DRL-based task offloading framework for UAV-assisted IoV networks, supporting local, V2V, and V2I modes through two enhanced DDPG networks for parallel optimization. RADiT improved task utility, energy efficiency, and delay performance. The same authors [21] later proposed a hybrid framework combining hierarchical asynchronous federated learning (HAFL) and multi-agent DRL (MADRL) to enhance task offloading efficiency across multiple modes, achieving higher task completion, lower delay, and better energy utilization. Similarly, Consul et al. [22] designed a federated reinforcement learning (FRL)-based hybrid offloading and resource allocation framework for DT-enabled UAV-MEC networks, which improved reward stability and reduced delay compared to baselines. Hevesli et al. [23] developed the Dynamic DT-Edge Air-Ground Network (D2TEAGN) for 6G IIoT, formulating a joint UAV trajectory, task offloading, and resource allocation problem (JUTIA-TORA) solved via DDPG. Although effective in reducing energy consumption and supporting real-time adaptability, D2TEAGN lacks U2U load balancing, limiting its performance under highly dynamic, latency-sensitive conditions.

Despite notable advancements, these studies share common limitations: the absence of task prioritization mechanisms, lack of dynamic simulation within DTs to reflect evolving system states, and reliance on static task models that fail to adapt to variable computational and energy constraints.

2.3. DRL-based optimization for UAV coordination

Deep Reinforcement Learning (DRL) and Multi-Agent DRL (MADRL) have shown considerable promise in tackling complex decision-making challenges in UAV-assisted edge computing environments. Unlike centralized approaches, MADRL enables decentralized agents (e.g., UAVs, IoT devices) to learn cooperative task offload and resource optimization strategies under dynamic and uncertain conditions.

In [25], Zhao et al. explored UAV-based task scheduling for smart city applications using DRL, highlighting the potential for adaptive, decentralized control. Expanding this direction, the work in [26] proposed a MADDPG-driven framework for joint service placement and task offloading in MEC-enabled Air-Ground Integrated Networks (AGINs), aiming to minimize long-term task delay and processing cost. Their approach effectively handles hybrid decision variables (binary, integer, continuous) under system constraints, making it suitable for time-critical missions such as disaster response. Qin et al. [27] further advanced multi-agent learning for air-ground vehicular cooperation by integrating MADDPG with Lyapunov optimization to satisfy ultra-reliable low-latency communication (URLLC) requirements while reducing energy-delay trade-offs. Their findings provide insights relevant to UAV-IoT collaboration, particularly for real-time, latency-sensitive operations.

The study in [28] reduced energy consumption and improved task offloading efficiency in UAV-assisted MEC networks through a collaborative MADRL strategy, where UAVs jointly learned energy-aware allocation and mobility policies under dynamic conditions. Similarly, [24] introduced a MADRL framework for hierarchical AEC systems with High Altitude Platforms (HAPs) and UAVs, integrating ConvLSTM-based workload prediction and prioritized experience replay with DQN to enable autonomous offloading using only local observations and without subtask division. However, these approaches do not explicitly address delay-sensitive, computation-intensive tasks, as they lack mechanisms for fine-grained prioritization and scheduling that jointly consider task urgency and resource availability-features essential for meeting stringent deadlines in large-scale AEC environments.

Table 1 presents a comparative summary of the developed GLEMATO framework against state-of-the-art methods. However, from the table and related studies discussion, our remark is that most of the

existing works focus on task offloading and resource optimization in multi-UAV-assisted AEC environments but often overlook key aspects such as fine-grained task prioritization and dynamic coordination through U2U communication to ensure workload balancing among AEC servers. Although some of the recent papers incorporate task priority with the help of MHA. Additionally, many methods support only binary offloading and lack DT integration for proactive decision-making. These limitations hinder their performance in large-scale, latency-sensitive AEC networks and have driven us to develop the GLEMATO framework, which integrates GTrXL-based attention with MADDPG-driven cooperative control. The gated memory mechanisms and positional encoding in GTrXL enable sharper distinction of high and low-priority tasks, particularly highlighting delay-sensitive and resource-critical ones, while the prioritized signals guide multi-agent learning for balanced server utilization and robust coordination. As a result, GLEMATO provides clearer prioritization, improved load distribution, and more efficient decision-making than existing approaches.

3. System model and assumptions

This section provides a detailed system model and an overview of the assumptions for computational task offloading in a complicated AEC network in our established GLEMATO framework.

We have assumed a natural disaster scenario where a base station has been damaged due to a natural calamity or disaster. As illustrated in Fig. 1, the proposed architecture comprises four segments: the local layer, the UAV layer, the DT model, and the cloud layer. Each IoT device $i \in I$ produces time-sensitive and computationally intensive tasks. A hierarchical architecture consisting of an airship with huge computation capability and a set of UAV or AEC server, $U = \{1, 2, 3, \ldots, u, u+1\}$ is deployed. Here, 1 to u is used to identify the AEC server with limited computation capability, and $u+1$ represents the Airship. The tasks generated from IoT devices can be divided into at most three portions. Initially, within the local layer consisting of IoT devices, a small portion of the computationally intensive generated task is computed locally according to the limited energy resources of the IoTs. The remaining portion is offloaded to a designated AEC server of the objective area in the AEC layer, which comprises several AEC servers. However, due to resource constraints or the need for load balancing among AEC servers, an AEC server may not be able to process the entire offloaded task. In such cases, it computes a partial portion and forwards the remaining part to a neighboring available AEC server $u' \in U \setminus \{u, u+1\}$ with the help of U2U communication or offloaded to the Airship $u+1$. This cooperative redistribution not only balances computational demand by delegating tasks from overloaded to underloaded AEC servers but also influences the overall latency and energy consumption of the system, as the transmission and execution delays driven by wireless channel quality, server queue status, and computational capability. Since AEC server trajectories directly impact system communication and computation performances, a realistic mobility model is essential. Conventional models such as Random Walk (RW), Random Waypoint (RWP), and deterministic trajectories either assume memoryless and abrupt motion or they lack adaptability, resulting in unstable communication links and poor performances. Therefore, we adopt the Gauss-Markov Mobility (GMM) model [29] to capture time-correlated UAV dynamics, enabling smooth trajectories and stable links for reliable task offloading within the AEC network environment.

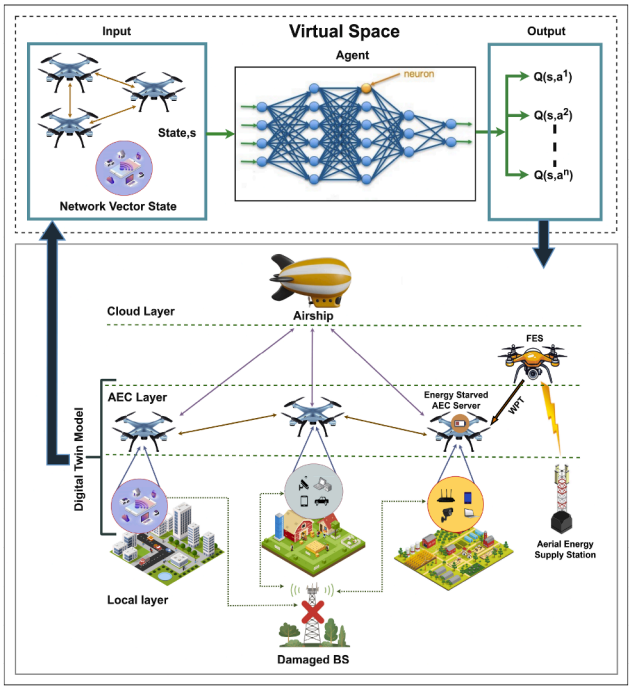


Fig. 1. A DT-enabled three-stage task offloading system in AEC environment.

Furthermore, if the charges of any AEC server become less than a threshold operational energy $E_u^{op}(t)$, it can harvest energy wirelessly with the assistance of Flying Energy Sources (FESs) [30], $F = \{1, 2, 3, \ldots, f\}$. Here, the number of FES is less than the number of AEC servers, i.e., $|F| < |U|$, and also each FES is capable of wireless recharging via an Aerial Energy Supply Station whenever required. Finally, to ensure proper load balancing and fairness among the AEC servers, we have assumed a threshold load balancing value $\sigma_u^{th}(t)$ to distribute tasks among the nearby AEC servers.

A DT model, deployed on the Airship, which establishes the virtual space for the local and AEC layers to monitor the real-time resource statuses and states of physical entities. The DT model $DT = \{(I, I'), (U, U')\}$ is a combination of DTs for IoTs and AEC servers and is employed at the Airship to collect real-time information about the status of physical entities. The DT of IoT device is denoted by $DT_i = \{C_i, \widehat{C}_i\}$, where C_i and $\widehat{C}_i$ indicate the estimated and deviation of computing resources between the real value in IoT device i and the value of its DT to compute the tasks locally in physical space, respectively. The DT of AEC server associated with IoT devices within a cluster denoted as $DT_u = \{C_{i,u}, \widehat{C}_{i,u}\}$, where, $C_{i,u}$ is the estimated computing resources allocated by the AEC server and $\widehat{C}_{i,u}$ is the deviation between the physical space and DT layers to compute the IoT device offloaded tasks [6]. Maintaining the fidelity of these resource estimates without excessive communication overhead is achieved via a Hybrid Synchronization Policy that manages the DT model update frequency. A periodic time-triggered update $\Delta t_{DT} = 10$ to 100 ms, ensures baseline freshness, while an error-triggered mechanism initiates an immediate update only when the DT deviation exceeds its tolerance threshold, $|\widehat{C}_i(t)| > \epsilon_{\text{IoT}}$ for IoT devices or $|\widehat{C}_{i,u}(t)| > \epsilon_{\text{AEC}}$ for AEC servers. This hybrid design maintains high model accuracy while significantly reducing unnecessary synchronization traffic. Table 2 summarizes the major list of notations to better understand the whole paper.

4. Design details of optimization framework

In this section, we detail the proposed MINLP-based optimization framework, which integrates a DT-enabled architecture and an AEC task-offloading model. The framework jointly optimizes task offloading and load balancing with the dual objectives of minimizing latency and energy consumption using a MINLP-based formulation.

4.1. Time-correlated 3D mobility model for AEC servers

To represent the dynamic nature of the network nodes, i.e., AEC servers and IoTs, the developed system is studied within a set T of $\{1, 2, 3, \ldots, t\}$ time slots. The system configuration is deemed static due to the brief duration of each time interval t.

Table 2
Major of notations.

Symbols	Description
I, U, F	Set of IoT devices, AEC servers and FESs
N	Set of sub-channels between IoT to AEC server
$\mathcal{B}^{n}_{i,u}$	Bandwidth between IoT device $i \in I$ and AEC server $u \in U$ using sub-channel $n \in N$
$p^{n,ul}_{i,u}(t), p^{n,dl}_{i,u}(t)$	Up-link and down-link transmission power between IoT device and AEC server
$R^{n,ul}_{i,u}(t)$	Achievable up-link data rate from IoT to AEC server over sub-channel at time slot t
$M_i(t), O_{i,u}(t)$	IoT device task data size and required computing resources to execute 1-bit of input task data size
$\Psi^{n,off}_{i,u}(t)$	Portion of task offloaded to the AEC server
$\Omega^{n,off}_{i,u' \vee u+1}(t)$	Portion of task offloaded to the neighbor AEC server u' or Airship $u+1$
$M^{post}_{i,u}(t)$	Size of the processed result of the IoT offloaded task
$C_i(t), C_{i,u}(t)$	Estimated CPU frequency for IoT device and AEC server
$d_{u \to u'}$	Euclidean distance between AEC servers
$\widetilde{\mathcal{T}}^{tot}_{i,u}(t), \widetilde{\mathcal{E}}^{tot}_{i}(t)$	Total normalized delay and energy consumption
$\mathcal{T}^{\max}_{i}(t), \mathcal{E}^{\max}_{i}(t)$	Maximum allowable delay and energy consumption for task computation
$\mathcal{L}_{i,u}(t), L_{i,u}(t)$	Normalized and instantaneous computation load in AEC server u at time slot t
$\sigma^{comp}_{u}(t)$	Standard deviation of the computational load of AEC server
$E^{rem}_{u}(t)$	Remaining energy capacity of AEC server u
$d^{\max}_{f,u}$	Maximum allowable distance between any AEC server u and FES f
$E^{harv}_{u,f}(t)$	Harvested energy of AEC server u from nearest FES f

Under this time-slotted framework, each AEC server $u \in U$ follows a Gauss-Markov Mobility model [29] with time-correlated kinematics. The integrated mobility state of AEC server u at time slot t is defined by the ordered tuple

$$\psi_u(t) = \left(\mathcal{V}^{fly}_{u}(t),\, \Theta_u(t),\, \phi_u(t)\right),$$

where the mobility state components $\mathcal{V}^{fly}_{u}(t)$, $\Theta_u(t)$, and $\phi_u(t)$ denote the instantaneous flying speed, azimuth angle, and elevation angle, respectively. The state components are updated independently as follows:

$$\psi_u(t) = \varpi \times \psi_u(t-1) + (1-\varpi) \times \bar{\psi}_u(t) + \sqrt{1-\varpi^2}\,\Pi_{\psi}(t). \tag{1}$$

Here, $\varpi \in [0,1]$ denotes the memory factor controlling temporal correlation, $\bar{\psi}_u(t)$ denotes the long-term mean of the mobility state, and $\Pi_{\psi}(t)$ is a zero-mean Gaussian random vector. Accordingly, the 3D position of each AEC server is compactly expressed as,

$$\varrho_u(t) = \varrho_u(t-1) + \mathcal{V}^{fly}_{u}(t) \times \begin{bmatrix} \cos\Theta_u(t)\cos\phi_u(t) \\ \sin\Theta_u(t)\cos\phi_u(t) \\ \sin\phi_u(t) \end{bmatrix} \Delta t, \tag{2}$$

where $\varrho_u(t) = \left[x_u(t),\, y_u(t),\, z_u(t)\right]^{\top} \in \mathbb{R}^{3\times 1}$ denotes the 3D cartesian position vector of AEC server u at time slot t, and Δt represents the duration of a time slot.

4.2. Air-to-ground communication model

To enable orthogonal spectrum allocation, we consider a set of sub-channels $N = \{1, 2, 3, \ldots, n\}$, where the bandwidth of each sub-channel $n \in N$ between IoT device to AEC server is $\mathcal{B}^{n}_{i,u}$. A binary variable $\delta^{n}_{i,u}(t) \in \{0,1\}$ is considered for the sub-channel assignment, which value is 1 if sub-channel n is assigned to IoT $i \in I$ associated with AEC server u to transmit data, otherwise, 0. By utilizing the free-space path loss model, the channel gain between IoT device i and AEC server $u \in U$ over sub-channel n is given by [31], [32]:

$$h^{n}_{i,u}(t) = \frac{h_0}{(d_{i,u})^{\varphi}}, \tag{3}$$

where h_0 is the channel gain at a reference distance of $1m$ and φ is the path loss exponent. And the Euclidean distance between IoT device i and AEC server u is denoted as, $d_{i,u} = \sqrt{(x_u - x_i)^2 + (y_u - y_i)^2 + (z_u - z_i)^2}$. The up-link signal to interference plus noise ratio ($SINR$) for the IoT device i associated with AEC server $u \in U$ over sub-channel n is presented as follows:

$$\upsilon^{n,ul}_{i,u}(t) = \frac{p^{n,ul}_{i,u}(t) \times h^{n}_{i,u}(t)}{\sum_{u' \in U, u' \neq u} \sum_{i' \in I, i' \neq i} \delta^{n}_{i',u'} \times P^{n,ul}_{i',u'}(t) \times h^{n}_{i',u'}(t) + N_0}, \tag{4}$$

where $p^{n,ul}_{i,u}(t)$ is the up-link transmission power over sub-channel n. N_0 and $h^{n}_{i,u}(t)$ are the spectral density of noise power and the channel gain over the sub-channel n between the IoT device and AEC server, respectively [31]. Here, we take into account the interference from other IoT devices associated with AEC server $u' \in U$, $u' \neq u$, transmitting over the same sub-channel n.

Hence, the achievable data rate for the up-link communication link from IoT device i associated with AEC server u over sub-channel n at each time slot t can be calculated using Shannon's theorem as follows:

$$R^{n,ul}_{i,u}(t) = \mathcal{B}^{n}_{i,u} \times \log_2\left(1 + \upsilon^{n,ul}_{i,u}(t)\right). \tag{5}$$

The achievable data rate for the down-link communication $R^{n,dl}_{i,u}(t)$ can be easily determined similarly by considering the down-link $SINR$ ratio $\upsilon^{n,dl}_{i,u}(t)$. The achievable up-link and down-link data rates between AEC servers and between an AEC server and the Airship follow the same calculation procedure used previously in Eqs. (3)–(5). Thus, without rederiving these expressions, the corresponding rates at time slot t are represented as $R^{n,ul}_{i,u\to u'}(t)$, $R^{n,dl}_{i,u\to u'}(t)$, $R^{n,ul}_{i,u\to u+1}(t)$ and $R^{n,dl}_{i,u\to u+1}(t)$.

4.3. Computational and service latency model

Each IoT device $i \in I$ has delay-sensitive computation task associated with AEC server $u \in U$, which can be defined as the tuple $\zeta_{i,u}(t) = \{M_i(t), O_{i,u}(t), \mathcal{T}^{\max}_{i}(t)\}$. Here, $M_i(t)$, $O_{i,u}(t)$ and $\mathcal{T}^{\max}_{i}(t)$ indicate the data size of the computing task, amount of required computing resources to execute 1-bit of input data and maximum tolerable delay for the completion of the task, respectively.

4.3.1. Local computation and service latency at IoT

Due to the limited energy and computing resources of IoTs, it is impossible to complete the tasks locally on time. The portion of the IoT task that will be executed locally at time slot t can be denoted as:

$$\mathcal{I}^{loc}_{i}(t) = \overbrace{\left(1 - \Psi^{n,off}_{i,u}(t)\right)}^{Local\ Processing} \times M_i(t), \tag{6}$$

where $\Psi^{n,off}_{i,u}(t) \in [0,1]$ is the task offloading ratio to the AEC server. When the value of $\Psi^{n,off}_{i,u}(t) = 1$, the task is fully offloaded to the AEC server and no local computation is performed.

The actual computational delay for task computation in the IoT can be calculated locally by combining the local computation delay $\widehat{T}^{loc}_{i}(t)$ and the difference between the local computation delay and DT

predicted delay $\Delta\widehat{T}_i^{loc}(t)$ as follows,

$$T_i^{loc}(t) = \widehat{T}_i^{loc}(t) + \Delta\widehat{T}_i^{loc}(t) = \frac{T_i^{loc}(t) \times O_{i,u}(t)}{C_i(t)} + \frac{T_i^{loc}(t) \times O_{i,u}(t) \times \widehat{C}_i(t)}{C_i(t)\Big(C_i(t) - \widehat{C}_i(t)\Big)}, \tag{7}$$

where $C_i(t)$ denotes the estimated CPU frequency of the IoT device and $\widehat{C}_i(t)$ indicates the deviation between the actual CPU frequency value of the IoT device i in the physical and DT layer at time slot t.

4.3.2. Computation offloading to AEC server and associated service latency

Due to the limited computing resources of IoT devices, $\Psi_{i,u}^{n,off}(t)$ portion of the tasks will be offloaded to their associated AEC servers. The transmission delay for the offloading task in AEC server $u \in U$ can be determined by:

$$T_{i,u}^{ul}(t) = \frac{\Psi_{i,u}^{n,off}(t) \times M_i(t)}{R_{i,u}^{n,ul}(t)}, \tag{8}$$

where $R_{i,u}^{n,ul}(t)$ denotes the achievable up-link communication link data transmission rate between IoT to AEC server.

Due to the limited buffer size to store IoT device's offloaded tasks, some tasks need to wait in the queue. The average queuing delay experienced by the AEC server can be estimated using the M/G/1 queuing model [33] as follows:

$$T_{i,u}^{que}(t) = \frac{\lambda_u \times \sigma_s^2 + \rho}{2 \times (1 - \rho)}, \tag{9}$$

where λ_u denotes the task arrival rate at $u \in U$ and σ_s^2 is the variance of service time. $\rho = \lambda_u / \mu_u$ denotes traffic intensity, μ_u denotes the service time and $\mu_u > \lambda_u$. The system utilization factor, as the traffic intensity plays a critical role in evaluating the performance of the AEC environment.

Sometimes AEC server can not compute all offloaded tasks due to resource constraints. In that case, it collaborates with the neighbor AEC servers or Airship and computes a portion of the offloaded task $\Psi_{i,u}^{n,off}(t) \times M_i(t)$, which can be estimated as follows:

$$\mathcal{U}_{i,u}^{cmp} = \overbrace{\Big(1 - \Omega_{i,u' \vee u+1}^{n,off}(t)\Big)}^{AEC\ Server\ Processing} \times \Psi_{i,u}^{n,off}(t) \times M_i(t), \tag{10}$$

where $\Omega_{i,u' \vee u+1}^{n,off}(t) \in [0,1]$ is the AEC server task offloading ratio to a neighboring AEC server or the Airship. The value of $\Omega_{i,u' \vee u+1}^{n,off}(t) = 1$ indicates that the remaining portion of the task is offloaded to a neighboring AEC server or to the Airship.

The actual computational delay to execute offloaded IoTs task in the AEC server can be expressed by combining the AEC server computation delay $\widehat{T}_{i,u}^{cmp}(t)$ and the difference between the AEC server computation delay and DT predicted delay $\Delta\widehat{T}_{i,u}^{comp}(t)$ in [6]:

$$T_{i,u}^{cmp}(t) = \widehat{T}_{i,u}^{cmp}(t) + \Delta\widehat{T}_{i,u}^{cmp}(t) = \frac{\mathcal{U}_{i,u}^{cmp}(t) \times O_{i,u}(t)}{C_{i,u}(t)} + \frac{\mathcal{U}_{i,u}^{cmp}(t) \times O_{i,u}(t) \times \widehat{C}_{i,u}(t)}{C_{i,u}(t)\Big(C_{i,u}(t) - \widehat{C}_{i,u}(t)\Big)}. \tag{11}$$

The down-link communication delay experienced by IoT device i associated with AEC server $u \in U$ at time slot t to receive the result can be calculated as:

$$T_{i,u}^{dl}(t) = \frac{M_{i,u}^{post}(t)}{R_{i,u}^{n,dl}(t)}, \tag{12}$$

where $M_{i,u}^{post}(t)$ denotes the size of the processed result of the offloaded task of IoT devices. Finally, the total delay incurred by the AEC server $u \in U$ can be calculated as:

$$T_{i,u}^{ex}(t) = T_{i,u}^{ul}(t) + T_{i,u}^{que}(t) + T_{i,u}^{cmp}(t) + T_{i,u}^{dl}(t). \tag{13}$$

4.3.3. Cooperative offloading to neighbor server and overall service latency

To prevents the collision between AEC servers, a threshold distance has been considered between them. The Euclidean distance between two servers can be expressed by considering the three-dimensional coordinates of AEC servers in their objective area as $d_{u \to u'} = \sqrt{(x'_u - x_u)^2 + (y'_u - y_u)^2 + (z'_u - z_u)^2}$ and must satisfy the threshold distance constraint $d^{u \to u'} \leq d_{u \to u'}^{th}$ to avoid collisions between two them in fixed-altitude.

The neighbor AEC Server or Airship will compute the remaining $\Omega_{i,u' \vee u+1}^{n,off}(t) \times \Psi_{i,u}^{n,off}(t) \times M_i(t)$ portion of the task. Therefore, the up-link transmission delay for the task to offload at the neighbor AEC server u' or the Airship $u+1$ is as follows:

$$T_{i,u' \vee u+1}^{ul}(t) = \Bigg[\overbrace{\Phi_i \times T_{i,u \to u+1}^{ul}(t)}^{Offload\ to\ Airship} + \overbrace{(1 - \Phi_i) \times T_{i,u \to u'}^{ul}(t)}^{Offload\ to\ Neighbor\ AEC\ Server}\Bigg] = \Bigg[\Phi_i \times \frac{\Omega_{i,u' \vee u+1}^{n,off}(t) \times \Psi_{i,u}^{n,off}(t) \times M_i(t)}{R_{i,u \to u+1}^{n,ul}(t)} + (1 - \Phi_i) \times \frac{\Omega_{i,u' \vee u+1}^{n,off}(t) \times \Psi_{i,u}^{n,off}(t) \times M_i(t)}{R_{i,u \to u'}^{n,ul}(t)}\Bigg], \tag{14}$$

where $\Phi_i \in \{0,1\}$ is the decision-making binary variable that will be 0, if the task is offloaded to the neighbor AEC server, otherwise, 1 for the airship. The queuing delay, task execution delay and down-link latency of neighboring AEC server or Airship denoted as $T_{i,u' \vee u+1}^{que}(t)$, $T_{i,u' \vee u+1}^{cmp}(t)$ and $T_{i,u' \vee u+1}^{dl}(t)$ can be calculated similar to Eqs. (9), (11) and (12), respectively. Therefore, the delay at the neighboring AEC server or the Airship is:

$$T_{i,u' \vee u+1}^{ex}(t) = T_{i,u' \vee u+1}^{ul}(t) + T_{i,u' \vee u+1}^{que}(t) + T_{i,u' \vee u+1}^{cmp}(t) + T_{i,u' \vee u+1}^{dl}(t). \tag{15}$$

Now, the normalized service delay $\widetilde{\mathcal{T}}_{i,u}^{tot}(t)$, is obtained by first computing the total service delay of an IoT task in three stages, $T_{i,u}^{tot}(t) = \Big[T_i^{loc}(t) + T_{i,u}^{ex}(t) + T_{i,u' \vee u+1}^{ex}(t)\Big]$, and normalizing it as follows,

$$\widetilde{\mathcal{T}}_{i,u}^{tot}(t) = \frac{T_{i,u}^{tot}(t)}{\mathcal{T}_i^{max}(t)}, \tag{16}$$

where $\mathcal{T}_i^{\max}(t)$ indicates the maximum allowable service delay of the task generated by IoT device i.

4.4. Coordination-aware load balancing model

To ensure balanced computation across AEC servers, we formulate a load balancing model that captures both instantaneous and historical server utilization by exploiting the Exponential Weighted Moving Average (EWMA) [34], as follows:

$$\mathcal{L}_{i,u}(t) = \alpha \times L_{i,u}(t) + (1 - \alpha) \times \mathcal{L}_{i,u}(t-1), \tag{17}$$

where $\mathcal{L}_{i,u}(t-1)$ is the normalized computational load at time slot $(t-1)$, and $L_{i,u}(t)$ is the instantaneous computational load of any AEC server u at time slot t consists of tasks generated by its associated IoT devices and the aggregated tasks offloaded from neighboring AEC servers. The weighted parameter $0 \leq \alpha \leq 1$ makes a balance between the old and new estimations.

$$L_{i,u}(t) = M_i(t) \times \Bigg[\overbrace{\sum_{i \in I} \Psi_{i,u}^{n,off}(t)}^{Associated\ IoTs\ Tasks} + \overbrace{\sum_{u \in U} \Big(\Omega_{i,u' \vee u+1}^{n,off}(t) \times \Psi_{i,u}^{n,off}(t)\Big)}^{Neighbor\ AEC\ Servers\ Offloaded\ Task}\Bigg]. \tag{18}$$

The global load imbalance across all servers is quantified using the standard deviation of computational loads as follows:

$$\sigma_u^{com}(t) = \sqrt{\frac{\sum_{u=1}^{U} \big(\mathcal{L}_{i,u}(t) - \bar{L}_{i,u}(t)\big)^2}{|U|}}, \tag{19}$$

where $\bar{L}_{i,u}(t)$ is the statistical mean of the computational loads of AEC server in time slot t. A lower value of $\sigma_u^{com}(t)$ indicates the even distribution of loads among the AEC servers.

4.5. Energy consumption model

In this section, we detail the calculations of energy consumption by the IoT devices (local layer), AEC servers, and neighbor AEC servers or Airship for executing the tasks.

4.5.1. Local energy consumption at IoT

The energy consumption of local computing by IoT devices can be estimated [35] as follows:

$$E_i^{loc}(t) = \mathcal{Q}_i(t) \times \left(C_i(t) - \widehat{C}_i(t)\right)^3 \times T_i^{loc}(t), \tag{20}$$

where $\mathcal{Q}_i(t)$ is the effective switching capacitance parameter related to the hardware capabilities of the IoT device.

4.5.2. Energy consumption in AEC server

The energy consumption for transmitting the data to the AEC server with an up-link communication link can be calculated as follows:

$$E_{i,u}^{ul}(t) = p_{i,u}^{n,ul}(t) \times T_{i,u}^{ul}(t), \tag{21}$$

where $p_{i,u}^{n,ul}(t)$ denotes the up-link transmission power to transmit the task from IoT to AEC server. The energy consumption by AEC server u to execute the partially offloaded tasks generated from IoT device i can be expressed as:

$$E_{i,u}^{cmp}(t) = \mathcal{Q}_u(t) \times \left(C_{i,u}(t) - \widehat{C}_{i,u}(t)\right)^3 \times T_{i,u}^{cmp}(t), \tag{22}$$

where $\mathcal{Q}_u(t)$ is the effective switching capacitance parameter related to the hardware capabilities of AEC server. The energy consumption for transmitting back the result to the IoT device from the AEC server is as follows:

$$E_{i,u}^{dl}(t) = p_{i,u}^{n,dl}(t) \times T_{i,u}^{dl}(t), \tag{23}$$

where $p_{i,u}^{n,dl}(t)$ is the down-link transmission power required for transmitting back the result from AEC server to IoT. Finally, the total computational and transmission energy consumption incurred by the AEC server can be calculated as:

$$E_{i,u}^{ex}(t) = E_{i,u}^{ul}(t) + E_{i,u}^{cmp}(t) + E_{i,u}^{dl}(t). \tag{24}$$

We assume that in each time slot, the AEC servers fly with an average flying speed $\bar{\mathcal{V}}_u^{fly}(t)$ for the flying duration $T_u^{fly}(t)$ seconds. Moreover, the AEC servers hover for IoT device association and compute offloaded tasks during communication and computation. The hovering energy consumption $E_u^{hov}(t)$ is considered as a constant similar to [36]. The flying energy consumption of a rotary-wing AEC server u with average flying speed $\bar{\mathcal{V}}_u^{fly}$ is calculated as follows [2],

$$E_u^{fly}(t) = T_u^{fly}(t) \times \bar{\mathcal{V}}_u^{fly}(t) \left[C_1 \times \left(\bar{\mathcal{V}}_u^{fly}(t)\right)^2 + \frac{C_2}{\left(\bar{\mathcal{V}}_u^{fly}(t)\right)^2} \right], \tag{25}$$

where the parameters C_1 and C_2 are determined by the AEC server's wing area, weight, and aerodynamic properties with capturing both parasitic and induced power components required to maintain flight. Finally, we can calculate the AEC server total energy consumption within its objective area of time slot t as follows:

$$E_{i,u}^{all}(t) = E_{i,u}^{ex}(t) + E_u^{hov}(t) + E_u^{fly}(t). \tag{26}$$

4.5.3. Energy consumption in the neighboring server and overall energy consumption

The up-link energy consumption for transmitting the remaining portion of the task to either neighboring AEC server $u' \in U \setminus \{u, u+1\}$ or to the Airship $u+1$, corresponding up-link transmission powers between AEC servers $p_{i,u\to u'}^{n,ul}(t)$ and from the AEC server $\to$ Airship $p_{i,u\to u+1}^{n,ul}(t)$, is calculated as follows:

$$\begin{aligned} E_{i,u'\vee u+1}^{ul}(t) &= \Phi_i \times E_{i,u\to u+1}^{ul}(t) + (1-\Phi_i) \times E_{i,u\to u'}^{ul}(t) \\ &= \Phi_i \times \left(p_{i,u\to u+1}^{n,ul}(t) \times T_{i,u\to u+1}^{ul}(t)\right) \\ &\quad + (1-\Phi_i) \times \left(p_{i,u\to u'}^{n,ul}(t) \times T_{i,u\to u'}^{ul}(t)\right). \end{aligned} \tag{27}$$

The overall energy consumption at either the neighboring server or the Airship $E_{i,u'\vee u+1}^{all}(t)$, similar to Eq. (26), accounts for the energy consumed in processing the remaining portion of the task, including execution, hovering, and flying energy consumptions during time slot t, and it can be defined as,

$$E_{i,u'\vee u+1}^{all}(t) = \begin{cases} E_{i,u'}^{ex}(t) + E_{u'}^{hov}(t) + E_{u'}^{fly}(t), & \text{if offloaded to } u', \\ E_{u+1}^{fly}(t), & \text{if offloaded to } u+1. \end{cases} \tag{28}$$

It is important to note that the execution and hovering energy consumptions at the Airship are neglected, since these components are task-independent and the Airship is assumed to have abundant computational and energy resources. Now, the normalized energy consumption $\widetilde{\mathcal{E}}_{i,u}^{tot}(t)$, is obtained by first evaluating the total energy consumption of an IoT task in three stages, $E_{i,u}^{tot}(t) = \left[E_i^{loc}(t) + E_{i,u}^{all}(t) + E_{i,u'\vee u+1}^{all}(t)\right]$ and subsequently normalizing it as:

$$\widetilde{\mathcal{E}}_{i,u}^{tot}(t) = \frac{E_{i,u}^{tot}(t)}{\mathcal{E}_i^{max}(t)}, \tag{29}$$

where $\mathcal{E}_i^{\max}(t)$ indicates the maximum allowable energy consumptions.

4.5.4. AEC server energy harvesting from FES

The AEC server can harvest energy from FES, and it continues its operations if the remaining energy is greater than or equal to a threshold operational energy, where the remaining energy can be calculated as:

$$E_u^{rem}(t) = E_u^{rem}(t-1) + E_{u,f}^{harv}(t) - E_{i,u}^{all}(t). \tag{30}$$

Here, $E_u^{rem}(t-1)$ denotes the energy level of AEC server in the previous time slot. At every time slot, AEC servers can harvest energy from the FESs wirelessly. The harvested energy $E_{u,f}^{harv}(t)$ from FES, which can be calculated as follows [30]:

$$E_{u,f}^{harv}(t) = \sum_{f=1}^{F} \sum_{u=1}^{U} \left(b_{f,u}^{chg}(t) \times \Gamma_{f,u}^{harv}(t) \times \left(\frac{d_{f,u}^{max}}{d_{f,u}}\right)^2 \times l_r \right), \tag{31}$$

where $b_{f,u}^{chg}(t)$ is a binary variable indicates whether AEC server u is being charged by FES f at time slot t or not, $\Gamma_{f,u}^{harv}(t) \in [0,1]$ represents the harvest-able energy ratio from FES, $d_{f,u}^{\max}$ and $d_{f,u}$ are the maximum allowable distance and current distance for energy harvesting between FES and AEC server, respectively. And l_r denotes the number of laser receptors on an AEC server. The energy harvesting from the FES through wireless communication is triggered only when the remaining energy of the AEC server falls below or is equal to the operational threshold, i.e., $E_u^{rem}(t) \le E_u^{op}(t)$.

4.6. Optimization framework

The objective function of the optimization framework to bring a trade-off between minimizing total service latency and energy consumption utilizing the MINLP problem in our three-stage AEC server-based task offloading architecture can be expressed as follows:

***Minimize*:**

$$\mathcal{J} = \underset{\{\delta_{i,u}^n(t), \varsigma_{i,u}^n(t), \Phi_i\}}{\arg\min} \sum_{t\in T} \sum_{i\in I} \sum_{u\in U} \{\gamma \times \widetilde{\mathcal{T}}_{i,u}^{tot}(t) + (1-\gamma) \times \widetilde{\mathcal{E}}_{i,u}^{tot}(t)\}, \tag{32}$$

where $\gamma \in [0,1]$ is a weighting factor, governs the trade-off between service latency and energy consumption within the objective function. When $\gamma = 1$, the framework exclusively prioritizes minimizing service latency; conversely, $\gamma = 0$ focuses on reducing energy consumption. For the intermediate values ($0 < \gamma < 1$), the optimization framework balances both objectives for optimal task offloading while adhering to service latency and energy consumption. Eq. (32) is subjected to the following constraints,

$$C_1 : \varsigma^n_{i,u}(t) \in \{0,1\}, \quad \forall n \in N, \forall i \in I, \forall u \in U$$
$$C_2 : \Psi^{n,off}_{i,u}(t), \Omega^{n,off}_{i,u' \vee u+1}(t) \in [0,1], \quad \forall i \in I, \forall u \in U$$
$$C_3 : \sum_{u \in U} \varsigma^n_{i,u}(t) \times T^{tot}_{i,u}(t) \leq \mathcal{T}^{max}_i(t), \quad \forall i \in I, t \in T$$
$$C_4 : \sum_{u \in U} \varsigma^n_{i,u}(t) \times E^{tot}_{i,u}(t) \leq \mathcal{E}^{max}_i(t), \quad \forall i \in I, t \in T$$
$$C_5 : \sigma^{com}_u(t) \leq \sigma^{th}_u(t), \quad \forall u \in U, t \in T$$
$$C_6 : E^{op}_u(t) \leq E^{rem}_u(t), \quad \forall u \in U, t \in T$$

$\varsigma^n_{i,u}(t)$ is a binary variable in constraint C_1 that governs whether an offloaded task of IoT device i is initially assigned to a particular AEC server u at time step t over sub-channel n or not. Constraint C_2 indicates the portion of the task offloading ratio from IoT → Associated AEC server, and Associated AEC server → Neighbor AEC server or Airship. Constraints C_3 and C_4 state that the total service delay and energy consumption to execute an assigned task $i \in I$ in time slot t must not exceed the maximum allowable delay $\mathcal{T}^{\max}_i(t)$ and the energy consumption $\mathcal{E}^{\max}_i(t)$, respectively. Constraint C_5 represents that the computational load of AEC servers must not exceed the threshold load $\sigma^{th}(t)$ to ensure proper load distribution. Constraint C_6 indicates that the remaining energy of AEC server u must be greater than the operational energy of AEC server to ensure smooth operation.

The optimal offloading decisions of computationally intensive tasks in a multi-UAV-assisted AEC network environment, while minimizing service delay and energy consumption, formulated in Eq. (32), is an NP-hard problem due to the complexity of conflicting constraints. The challenge is also exacerbated by various complex interdependent constraints related to communication bandwidth, computing capabilities, task and energy consumption deadlines, as well as the dynamic nature of networks.

Theorem 1. *The optimization problem to jointly minimize service delay and energy consumption, considering load balancing issues in a multi-UAV-assisted AEC network environment, as defined in Eq. (32), is an NP-hard problem.*

Proof.

The optimization framework formulated using MINLP has two conflicting objectives, i.e., minimizing service delay and energy consumption for offloading computationally intensive IoT tasks to the AEC network. Such an optimization framework is classified as an NP-hard problem and cannot be solved in polynomial time for large-scale networks. This formulated MINLP problem can be reduced to the well-known **G**eneralized **A**ssignment **P**roblem (**GAP**) as an NP-complete problem [37]. The GAP has the following components:

- A set $\mathcal{K} = \{1, \ldots, k\}$ of tasks.
- A set $\mathcal{G} = \{1, \ldots, g\}$ of agent.
- The cost function, C represents the associated expense for assigning a task $k \in \mathcal{K}$ to an agent $u \in \mathcal{U}$.
- A capacity function, Υ that provides the available capacity of an agent $g \in \mathcal{G}$.
- An availability function $\mathcal{Y}$ that indicates the available capacity of an agent $g \in \mathcal{G}$.

This problem targets to minimize the overall cost associated with executing jobs of all agents.

Minimize:

$$\arg\min_{\{x,u,q\}} \sum_{k \in \mathcal{K}} \sum_{g \in \mathcal{G}} C_{k,g} \times \mathcal{X}_{k,g}, \tag{33}$$

Subject to:

$$\mathcal{X}_{k,g} \in \{0,1\}, \quad \forall k \in \mathcal{K}, \forall g \in \mathcal{G} \tag{34}$$
$$\sum_{g \in \mathcal{G}} \Upsilon_k \mathcal{X}_{k,g} \leq \mathcal{Y}_g, \quad \forall k \in \mathcal{K} \tag{35}$$
$$\sum_{g \in \mathcal{G}} \mathcal{X}_{k,g} \leq 1, \quad \forall k \in \mathcal{K} \tag{36}$$

In a multi-UAV-assisted AEC network environment, the optimal task offloading problem can be reduced to GAP by relaxing some constraints. Let $\mathcal{W}_{i,u}(t) = (1-\gamma) \times \widetilde{\mathcal{E}}^{tot}_{i,u}(t)$. By considering the energy consumption of computational task execution of IoT devices and AEC servers, the optimization framework can be reduced in the following form,

Minimize:

$$\arg\min_{\{\delta^n_{i,u}(t), \varsigma^n_{i,u}(t), \Phi_i\}} \sum_{i \in I} \sum_{u \in U} \mathcal{W}_{i,u}(t) \times \varsigma^n_{i,u}(t), \tag{37}$$

Subject to:

$$\varsigma^n_{i,u}(t) \in \{0,1\}, \quad \forall i \in I, \forall u \in U \tag{38}$$
$$\sum_{u \in U} \varsigma^n_{i,u}(t) \times E^{tot}_{i,u}(t) \leq \mathcal{E}^{max}_i(t), \quad \forall i \in I, t \in T \tag{39}$$
$$\sum_{n \in N} \varsigma^n_{i,u}(t) \leq 1, \quad \forall i \in I, \forall u \in U, \forall t \in T \tag{40}$$

As the aforementioned MINLP-based task offloading problem in the AEC environment can be reduced to GAP, therefore, we can presume that, for large-scale AEC servers-assisted networks, the complexity of this formulation is at least as hard as GAP. Consequently, the proposed problem is classified as NP-hard in nature and cannot be solved in polynomial time. □

5. Gated transformer enhanced MADDPG algorithm

To resolve the NP-hardness of the formulated optimization problem in large-scale AEC networks, we develop GLEMATO, a GTrXL-assisted MADDPG framework that learns high-quality offloading policies through cooperative multi-agent training rather than relying on exact polynomial-time optimization. By combining memory-aware task prioritization with decentralized policy updates, the GLEMATO provides a practically scalable and adaptive approximation strategy that remains effective under dynamic and uncertain AEC conditions, enabling robust task scheduling and resource coordination across multiple UAVs.

5.1. MADDPG For optimal task offloading decision

Reinforcement Learning (RL) is grounded in the Markov Decision Process (MDP), defined by state, action, reward, transition dynamics, and a discount factor [38]. While classical RL methods enable effective single-agent decision-making, they struggle with high-dimensional and continuous action spaces. Deep Reinforcement Learning (DRL) mitigates these limitations by leveraging deep neural networks and replay buffers for stable function approximation [9]. The Deep Q-Network (DQN) introduced a major breakthrough, yet remains restricted to discrete actions. Its extension, Multi-Agent DQN (MADQN), employs individual Q-networks to support autonomous and cooperative behavior in complex, dynamic, and multi-agent environments [30]. To address continuous action spaces, the Deep Deterministic Policy Gradient (DDPG) framework utilizes actor-critic networks and experience replay. However, its performance degrades in multi-agent settings due to limited exploration and scalability issues [6]. The Multi-Agent DDPG (MADDPG) algorithm overcomes these challenges through centralized training with decentralized execution, allowing agents to exploit joint state-action information during training while acting independently at deployment [26,30]. This

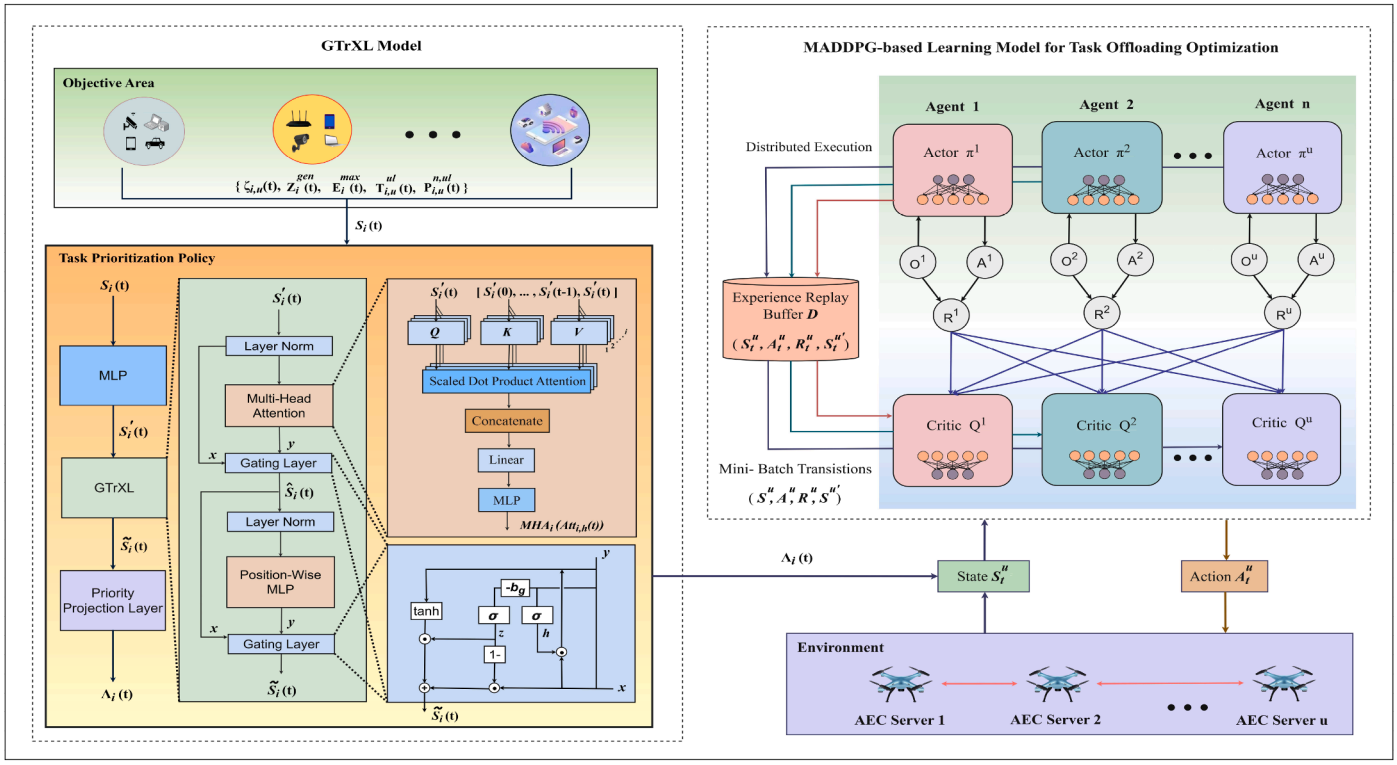


Fig. 2. Workflow of the gated transformer MADDPG algorithm for optimal task offloading strategy in a multi-UAV-assisted AEC environment.

structure supports coordinated learning, individualized decision policies, and robust performance in dynamic environments. Building on this foundation, the developed GLEMATO framework enhances MADDPG to effectively handle cooperation, prioritization, and resource-aware decision-making among multiple interacting AEC servers.

The GLEMATO framework improves task offloading decisions and resource utilization by enabling U2U-based collaboration among AEC servers, allowing them to share task information, resource states, and environmental context in real time. This distributed awareness supports intelligent offloading and dynamic load balancing, ensuring tasks are directed to the most suitable agents while reducing delay and energy consumption. Aligned with MADDPG principles, GLEMATO further enhances inter-agent cooperation for efficient handling of complex workloads.

5.2. Tasks prioritization mechanism with gated transformer-XL model

The Multi-Head Attention (MHA) mechanism used in standard transformers [39] is effective for capturing short-term feature relevance but becomes less reliable when modeling long-range sequential dependencies, especially in dynamic AEC-assisted IoT networks where task characteristics and resource availability evolve continuously [40]. The GTrXL model addresses these limitations by introducing recurrent connections that strengthen long-term dependency modeling and better reflect the sequential nature of IoT task patterns [41]. It integrates Multi-Head Self-Attention with a GRU-inspired gating structure, allowing attention to capture diverse contextual relationships while the gating mechanism emphasizes critical features and suppresses irrelevant fluctuations. This combination reduces computational overhead, yields more stable and discriminative priority scores between high and low priority tasks scores from wide-range sequential features dependencies compared to MHA, and ultimately improves MADRL learning efficiency for intelligent task offloading in complex multi-agent environments.

Fig. 2 illustrates the proposed GTrXL-enhanced MADDPG framework, which integrates GTrXL for task prioritization and MADDPG for cooperative decision-making among AEC servers (agents). The left portion of the figure highlights the role of the GTrXL model in computing task priority scores for IoT devices based on various task-specific attributes. These scores help AEC servers to identify and offload the most critical tasks in real-time. The GTrXL uses an MLP encoding layer initially to produce feature embeddings from wide-range input features, subsequently refined through the GTrXL block comprising MHA, GRU-based gating layers, and positional encoding strategies to preserve temporal context and emphasize relevant parts of the inputs depending on the current task. Initially, a multi-layer perception MLP(·) is used to embed the observation $S_i(t)$ is the task information before offloading, and transforms into a latent state variable $S_i^{'}(t)$ to form an input for MHA. In particular, the observation vector $S_i(t)$, which represents the state of task i at time t, includes several task requirement parameters such as, latent task profile $\zeta_{i,u}(t)$, consisting of task size, computation required per bit, and maximum tolerable delay, task generation time $\mathcal{Z}_i^{gen}(t)$, maximum allowable energy consumption to compute tasks of IoT devices, $\mathcal{E}_i^{\max}(t)$, up-link transmission delay, $T_{i,u}^{ul}(t)$ and up-link transmission power $p_{i,u}^{n,ul}(t)$, at time slot t are considered as observation $S_i(t)$.

$$S_i(t) = \{\zeta_{i,u}(t), \mathcal{Z}_i^{gen}(t), \mathcal{E}_i^{max}(t), T_{i,u}^{ul}(t), p_{i,u}^{n,ul}(t)\}. \tag{41}$$

These features are transformed into a latent representation using the MLP denoted as, $S_i^{'}(t) = MLP\left(S_i(t)\right) \in \mathbb{R}^{d_{model}}$, which not only reduces dimensionality but also captures nonlinear feature interactions. The MHA module in the GTrXL model employs parallel-scaled dot-product attention heads over each IoT device's task i feature embeddings. To capture temporal dependencies across task states, each attention head $h = \{1, \ldots, H\}$ applies learnable projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$ that map task embeddings into query, key, and value spaces. The current task embedding $S_i'(t)$ is projected into a query $Q_{i,h}(t) = W_Q^h S_i'(t)$, while all past embeddings $[S_i^{'}(0), \ldots, S_i^{'}(t-1), S_i^{'}(t)]$ are mapped to keys and values: $K_{i,h}(t) = W_K^h S_i'(t-1)$, and $V_{i,h}(t) = W_V^h S_i'(t-1)$.

This facilitates effective temporal attention modeling over past task states, improving task selection for offloading. Each attention head independently extracts different aspects of the task context using its

projection matrices, and the temporal context for any IoT device i task at timestep t is computed by applying Softmax-normalized scaled dot-product attention over all previous states and aggregating their Value vectors:

$$Att_{i,h}(t) = \sum_{n=0}^{t} Softmax\left(\frac{Q_{i,h}(t) \cdot K_{i,h}(t)^{\top}}{\sqrt{d_i}}\right) V_{i,h}(t), \tag{42}$$

where d_i is the input dimension of each IoT device task. After computing the head-specific temporal contexts, the MHA module concatenates the outputs of all heads and applies a linear projection, producing a fixed-size representation that integrates the parallel scaled dot-product attention outputs for downstream processing:

$$MHA_i(t) = Concat\Big(Att_{i,1}(t), \ldots, Att_{i,H}(t)\Big) \in \mathbb{R}^{d_{out}}. \tag{43}$$

The concatenated output feeds forward in an MLP layer as the output of the MHA module, as MHA(.), to generate the attention score of individual tasks of IoT devices. Also, a gating layer denoted as Gate(.,.), is incorporated to enable each AEC server to extend its focus and selectively prioritize specific task attention scores from the long-range task distribution. Inspired by GRU, we adapt its powerful gating mechanism into an untied activation function to stabilize temporal features and control information flow. It employs reset and update gates, along with a candidate state, to merge past information with new task-attention features.

$$z = \sigma(W_1 y + W_2 x - b_g), \quad h = \sigma(W_3 y + W_4 x),$$

$$\text{Gate}(x, y) = (1 - z) \odot x + z \odot \tanh(W_5 y + W_6(h \odot x)),$$

where W and b_g denote different embedding weights and gating bias, which control the identity initialization, respectively. The forward propagation in each block is calculated by:

$$\begin{aligned} \hat{S}_i(t) &= Gate\Big(S_i'(t), MHA_i(t)\Big), \\ \tilde{S}_i(t) &= Gate\Big(\hat{S}_i(t), MLP\big(\hat{S}_i(t)\big)\Big). \end{aligned} \tag{44}$$

To obtain an absolute urgency score for each task, the final vector embedding of GTrXL $\tilde{S}_i(t)$, passed through a sigmoid-activated linear projection, and the resulting priority scores are incorporated into the MADDPG state space,

$$\Lambda_i(t) = \sigma\Big(w_p^{\top} \tilde{S}_i(t) + b_p\Big) \in [0, 1], \tag{45}$$

where w_p and b_p are learnable parameters. To ensure reproducibility and to provide a mathematically explicit definition of task urgency, each task is also assigned a ground-truth target label $\hat{\Lambda}_i(t)$ derived from three key factors: (i) deadline pressure, (ii) task size, and (iii) remaining energy of the associated AEC server. The target priority is computed as,

$$\hat{\Lambda}_i(t) = \text{clip}\left(\beta_1\left(1 - \frac{\Delta_i(t)}{\mathcal{T}_i^{max}(t)}\right) + \beta_2 \frac{M_i(t)}{M_{i,max}^{size}} + \beta_3\left(1 - \frac{E_u^{rem}(t)}{\mathcal{E}_i^{max}(t)}\right),\ 0,\ 1\right), \tag{46}$$

where $\Delta_i(t)$ is the remaining deadline slack. The coefficients ($\beta 1, \beta 2$, and $\beta 3$) weight the contributions of temporal urgency, computational load, and energy awareness, respectively, and satisfy $\beta_1 + \beta_2 + \beta_3 = 1$. To ascertain a continuous priority regression and learn fine-grained urgency patterns from temporal attention, the GTrXL priority head is trained using Mean Squared Error (MSE) as follows,

$$L_{\text{pr}}(t) = \frac{1}{|\mathcal{N}|} \sum_{i \in \mathcal{N}} \left\| \hat{\Lambda}_i(t) - \Lambda_i(t) \right\|_2^2. \tag{47}$$

5.3. MADDPG components for optimal task offloading in an AEC environment

The MADDPG model, which forms the backbone of the GLEMATO framework, operates in a distributed multi-agent reinforcement learning paradigm where each AEC server acts as an agent, which is deployed in a specific objective area with hovering at a fixed altitude. The AEC server-enabled task-offloading-based MADDPG algorithm is modeled as an MDP, which is effectively described by a triad of components: state space, action space, and reward function. A detailed description of the above components of the developed MADDPG on the GLEMATO environment is given below.

- **State Space:** The state space encompasses various key components that collectively capture the overall status of the environment that each AEC server (agent) considers when executing the IoT devices task. Formally, the state space of each agent $u \in U$ is defined as:

$$S_t^u = \{\Lambda_i(t), E_u^{rem}(t), E_u^{op}(t), C_{i,u}(t)\}, \tag{48}$$

  where $\Lambda_i(t)$ represents the tasks priority score of GTrXL model from Eq. (45). $E_u^{rem}(t)$ and $E_u^{op}(t)$ indicate the remaining and operational energy required to calculate the assigned tasks of each AEC server $u \in U$. The estimated task computation capacity of each AEC server is also represented as $C_{i,u}(t)$.
- **Action Space:** The action taken by any agent is represented as a vector, denoted as $a \in A$, which is comprised of a set of decisions it can make within the system, guiding the behavior and operations of the AEC servers in the system. Based on the environment's state at time slot t, the action space for agent $u \in U$ is defined as:

$$A_t^u = \{a_t^{off}, a_t^{load}\}, \quad \forall a \in A, u \in U, t \in T \tag{49}$$

  where a_t^{off} is the offloading ratio to execute the task immediately, represented as $a_t^{off} = \{\Psi_{i,u}^{n,off}(t), \Omega_{i,u' \vee u+1}^{n,off}(t)\}$, and $a_t^{load} = \{\Omega_{i,u' \vee u+1}^{n,off}(t) \rightarrow \arg\min_{u' \in U} L_{i,u'}(t)\}$ denotes the load balancing decision which evaluates the AEC servers current load and offload task to least-loaded neighbor AEC server u' for balanced computation.
- **Reward Function:** At each time slot t, each agent $u \in U$ takes an action in the environment and moves to a new state $S_t^{u'}$, and a reward, $\mathcal{R}_t^u$ is received. The reward function is negatively correlated with the objective function formulated in Eq. (32), so maximizing the reward values naturally minimizes the service delay and total energy consumption, and it's defined as follows:

$$\mathcal{R}_t^u = -\Big(\gamma \times \mathcal{T}_{i,u}^{tot}(t) + (1 - \gamma) \times \mathcal{E}_{i,u}^{tot}(t)\Big). \tag{50}$$

  The agents receive a positive reward if they operate within the constraints, such as task delay limits, the energy consumption bounds, AEC server operational energy thresholds, and the computational loads capacities. Conversely, a penalty denoted as $\mathcal{P}_t^u$ is triggered if any of these constraints are violated by the agents.

5.4. Gated transformer MADDPG algorithm for the AEC environment

The GLEMATO framework builds on the MADDPG algorithm to enable cooperative and adaptive task offloading among AEC servers, as outlined in Algorithm 1. Lines 1-4 initialize key hyperparameters, including episodes (EPS), time steps (T), exploration noise (ξ), and replay buffer capacity (D). Lines 5-6 set up actor and critic networks, along with their target networks, where the actor selects actions, the critic evaluates them, and target networks stabilize training. Importantly, GTrXL-based task prioritization $\Lambda_i(t)$ of each node from Eq. (48) is embedded into the state space to reinforce the learning process of the MADDPG algorithm. The training process, spanning lines 7 to 30, begins with an episode loop from 1 to EPS. At the start of each episode in line 8, the environment is reset, and the initial state of each AEC server is established. For every time step t, an augmented state S_t^u is constructed on line 10, and actions A_t^u are selected using the current policy with exploration from lines 11-13. If the system incurs penalty $\mathcal{P}_t^u$, the corresponding action is discarded and the state is adjusted accordingly. Now, in lines 18 and 19, the reward value is calculated, after which in line 20, we observe the reward $\mathcal{R}_t^u$ and the new state $S_t^{u'}$. Then, the current state, action, reward, and next state are stored in the replay buffer D_n on line

21. Storing this information in the experience replay buffer is essential, as it enables the GLEMATO algorithm to learn from past experience, which enhances the training process. In the update stage (lines 22-28), agents sample mini-batches from $\mathcal{D}_n$ to refine policies. The critic network is updated by minimizing loss between estimated target Q-values by using the Bellman equation, while the actor network is optimized using policy gradients. Finally, soft updates of target networks are applied with factor τ, ensuring stable convergence.

To ensure that the decentralized MADDPG solution consistently adheres to the MINLP constraints (C_1-C_6) defined in Eq. (32), feasibility is enforced at two critical points in the learning loop-during action generation and during reward evaluation. Specifically:

- **Action Space Bounding:** The association and offloading decision bounds (C_1-C_2) are enforced directly through the actor's output layer, where bounded activations and discretizations ensure all actions remain within the permissible range.
- **Dynamic Penalty Shaping:** Resource and capacity limits (C_3-C_6) are enforced through a dynamic penalty mechanism, where any violation triggers an immediate penalty $\mathcal{P}_t^u$ and negative reward, guiding the policy toward the MINLP-feasible region.

5.5. Computational complexity of GLEMATO

The overall complexity of the developed GTrXL-driven MADDPG algorithm is inherently dependent on the structural complexity of both the transformer layers of GTrXL and the actor and critic networks of the MADDPG algorithm. This complexity is further influenced by the number of agents in action, as well as the layers within the neural networks. The computational complexity of the algorithm can be formally expressed by:

$$O\Bigg(\overbrace{L_g \cdot \left(T \cdot d_{model}^2 + T^2 \cdot d_{model}\right)}^{GTrXL} + \underbrace{2N \cdot \left(\sum_{u=0}^{L_a-1} n_u^a \cdot n_{u+1}^a + \sum_{p=0}^{L_c-1} n_j^c \cdot n_{j+1}^c\right)}_{MADDPG}\Bigg), \tag{51}$$

where the first term corresponds to the complexity of the GTrXL. Here, T represents the length of the input sequence of historical task states for GTrXL, and d_{model} is the embedding dimension of the model, and L_g indicates the number of transformer layers, which reflects the cost of self-attention and feedback actions in each GTrXL layer over multiple agents. The second term captures the complexity of the actor and critic network in the MADDPG algorithm, where L_a and L_c denote the number of layers in the actor and critic network, respectively. Here, N indicates the number of agents throughout the system. In this context, n_u^a and n_{u+1}^a are the dimensions of the input and output for the u_{th} actor network layer, while n_j^c and n_{j+1}^c refer to the input and output dimensions of the j_{th} critic network layer, respectively [42].

6. Performance evaluation

To provide a fair, representative, and technically meaningful comparative evaluation of the GLEMATO framework under diverse decision-making paradigms, we have selected four state-of-the-art baseline algorithms in UAV-assisted MEC offloading that collectively cover the key methodological families: (i) learning-free optimization without DT, (ii) single-agent RL with DT, (iii) predictive recurrent RL without DT, and (iv) multi-agent RL with DT. GTCO [13] provides a game-theoretic, learning-free reference for decentralized optimization-driven decision making. DQN [9] serves as a canonical single-agent, value-based DRL baseline in DT-enabled UAV settings without cooperative policy support. ConvLSTM-PER-DQN [24] offers a multi-agent cooperative behavior by integrating spatio-temporal workload forecasting via ConvLSTM

Algorithm 1 Gated transformer MADDPG algorithm.

1: $EPS \leftarrow$ Number of episodes
2: $T \leftarrow$ Number of time steps per episode
3: $\xi \leftarrow$ Action noise for exploration
4: **Initialization:** Replay buffer with maximum capacity $\mathcal{D}$
5: **Initialization:** Actor-network $\pi_u(\cdot)$ and critic-network $Q_u(\cdot)$ with their respective weights θ^{π_u} and θ^{Q_u} randomly.
6: **Initialization:** Target actor $\pi_{u'}(\cdot)$ and critic $Q_{u'}(\cdot)$ networks with their respective weights $\theta^{\pi_{u'}}$ and $\theta^{Q_{u'}}$
$\theta^{Q_{u'}} \leftarrow \theta^{Q_u}$, $\theta^{\pi_{u'}} \leftarrow \theta^{\pi_u}$
7: **for** episode $\leftarrow 1$ **to** EPS **do**
8: Initialize location environment and state S_0^u of $AEC\ server$
9: **for** $t \leftarrow 0$ **to** T **do**
10: **for** agent $u \in U$ **do**
11: Construct augmented state for AEC server u:
$S_t^u = \{\Lambda_i(t), E_u^{rem}(t), E_u^{op}(t), C_{i,u}(t)\}$
12: Select action for each AEC server u based on current policy π_u:
$A_t^u = \pi_u\left(\mathcal{O}_t^u \mid \theta^{\pi_u}\right) + \xi$,
// w.r.t. the current policy and exploration noise ξ
13: Execute action A_t^u
14: **end for**
15: **if** Penalty $\mathcal{P}_t^u > 0$ **then**
16: Cancel current action A_t^u and update state $S_t^{u'}$
17: **end if**
18: Get reward $\mathcal{R}_t^u$ based on Eq. (50), and adjust penalty:
19: $\mathcal{R}_t^u \leftarrow \mathcal{R}_t^u - \mathcal{P}_t^u$
20: Observe reward $\mathcal{R}_t^u$ and new state $S_t^{u'}$
21: Store $(S_t^u, A_t^u, \mathcal{R}_t^u, S_t^{u'})$ in experience replay buffer $\mathcal{D}_n$
22: **for** agent $u \leftarrow 1$ **to** N **do**
23: Sample random mini-batch of transitions $(S^u, A^u, \mathcal{R}^u, S^{u'})$ from $\mathcal{D}_n$
24: Set target network value as:
$y_t^u = \mathcal{R}_t^u + \gamma Q_{u'}\left(S_t^{u'}, \pi^{u'}(\mathcal{O}_t^{u'} \mid \theta^{\pi_{u'}}) \mid \theta^{Q_{u'}}\right)$
25: Update the main critic network by minimizing loss with:
$L(\theta^{Q_u}) = \frac{1}{|U|}\sum_{u=1}^{U}\left(y_t^u - Q_u(S_t^u, A_t \mid \theta^{Q_u})\right)^2$
26: Update the main actor network with policy gradient:
$\Delta_{\theta^{\pi_u}} J = \frac{1}{|U|}\sum_{u=1}^{U} \Delta_{A_t^u} Q^{\pi_u}(S_t^u, A_t^u \mid \theta^{Q_u}) \Delta_\theta \pi_u(\mathcal{O}_t^u)$
27: Update actor and critic target network parameter:
$\theta^{Q_{u'}} \leftarrow \tau\theta^{Q_{u'}} + (1-\tau)\theta^{Q_{u'}}$
$\theta^{\pi_{u'}} \leftarrow \tau\theta^{\pi_{u'}} + (1-\tau)\theta^{\pi_{u'}}$
28: **end for**
29: **end for**
30: **end for**

representations, though without DT-assisted collaboration, making it a strong predictive baseline for comparison. D2TEAGN [6], developed for 6G air-ground integrated networks, enables joint optimization of trajectory, association, and offloading, aligning closely with real-time, multi-stagedecision UAV-AEC deployments. All frameworks were assessed using the same hyperparameter configurations, as presented in the Table 4, on OpenAI Gym v0.26.2 platform [43] to guarantee fair and consistent comparison. The simulation experiments were executed on Windows 10 64-bit with an AMD Ryzen 5 3500X 3.60 GHz CPU, coupled with an NVIDIA GeForce GTX 1650 Super GPU 4 GB GDDR6, an MSI B450M MORTAR MAX motherboard, and 16 GB RAM as hardware.

6.1. Simulation environment

To evaluate the proposed GLEMATO framework, we simulate a disaster-affected, infrastructure-deficient region spanning approximately 6.1km × 19.4km, where 3-15 AEC servers operate cooperatively with 200-500 heterogeneous IoT devices and 2-5 FESs to ensure reliable, real-time communication and task offloading. This environment reflects

Table 3
Simulation parameters.

Parameter Descriptions	Assigned Values
Area of the Environment	6.1 km × 19.4 km
Number of AEC servers	3 – 15
Number of IoT devices	200 – 500
Tasks generate per devices	2 – 10
Number of FESs	2 – 5
Maximum Distance of AEC server from FESs	500 m
Safe Spacing Between AEC servers	50 – 100 m
Channel Bandwidth Allocation	5 – 20 MHz
Maximum Travel Speed of an AEC server	10 m/s
Task Data Size	10 – 500 MB
Application Delay Deadline	500 – 3500 ms
Computational Capacity of IoT devices	0.5 – 1.5 GHz
Computational Capacity of AEC servers	2.5 – 4.2 GHz
Average Energy Consumption of IoT Devices	0.5 – 1.5 J/task
Average Energy Consumption of AEC servers	2.0 – 4.5 J/task
Energy Capacity of an AEC server	500 kJ
Energy Capacity of an FES	1000 kJ
Operational Energy Consumption of an AEC server	50 kJ
Offloading Energy Consumption of an AEC server	25 J/task
Energy Harvesting Efficiency	0.5 – 5 J/s
AEC server's Energy Harvesting Threshold	20% – 30%

the urgent needs of search-and-rescue and emergency response scenarios, where conventional terrestrial communication infrastructure is either unavailable or compromised [1], [10]. The AEC servers maintain a flying speed of up to 10 m/s, ensuring operational coverage area through swarm coordination within 300–500 m for air-to-ground and 300–800 m for air-to-air links, while operating inside a 500 m range of the FESs, and keep a 50-100 m safe distance from each other. Environmental constraints include urban obstacles, varying terrain, multi-path fading, channel interference, and adverse weather effects, all of which impact wireless communication, AEC server maneuverability, and sensor reliability. The simulation environment incorporates a dynamic sub-channel allocation mechanism and terrain-aware signal propagation models. To emulate real-time service operations, 2-10 tasks are generated over a 4-minute window in random time slots, enabling asynchronous and event-driven task behavior.

To evaluate system performance under the defined constraints, a synthetic dataset [1] has been generated by constructing spatial IoT clusters with realistic metropolitan dispersion using a reproducible Python pipeline (seed = 42), and sampling device coordinates from Gaussian distributions around each cluster center consistent with an AEC server. The attributes of the dataset were uniformly distributed and sampled using empirically grounded 5G IoT models, while adjustments of controlled anomalies were injected to mimic real-world irregularities with noise in the AEC-assisted IoT network topology. The dataset validation was performed through spatial clustering checks, distribution analysis of all sampled features, physics-consistent channel and rate calculations, and strict enforcement of parameter bounds to ensure realistic and reliable data quality. The final dataset contains 5000 computationally intensive task samples from 526 IoT devices with 29 features, including task type, size, deadline, device-server distance, channel bandwidth, gain, data rate, energy states, and GTrXL-generated priority scores. Task CPU demands span 0.5-1.5 GHz for IoT devices and 2.5-4.2 GHz for AEC servers, while channel bandwidths vary between 5-20 MHz [44] under a uniform random distribution. We assume the AEC servers have an energy capacity of 500 kJ, while FESs hold up to 1000 kJ. Task execution on AEC servers costs approximately 50 kJ per cycle, and each offloaded task consumes about 25 J. Energy harvesting at FESs ranges between $0.5 \sim 5J/s$, with replenishment mechanisms triggered when energy levels fall below 20–30%. All reported performance metrics are averaged

[1] The dataset, ***AEC-IoT TaskNet*** is available at Kaggle repository. This allows for reproducible research and further analysis by the community.

Table 4
Hyper-parameters.

Parameter	Value
Number of episodes, EPS	3000
Number of time steps per episode, T	100
Memory replay buffer size, D	200,000
Actor Learning Rate	5×10^{-6}
Critic Learning Rate	1×10^{-5}
Mini-batch size	256
Noise rate for action exploration	0.2
Discount factor, γ	0.99
Optimizer for actor network	Adam
Optimizer for critic network	Adam
Critic weight decay	1×10^{-2}
Soft update rate, τ	0.0005

Table 5
Configuration of actor and critic networks.

Parameter Description	Actor Network	Critic Network
Number of Input Layers	1	1
Number of Hidden Layers	2	2
Number of Output Layers	1	1
Size of First Input Layer	$d_s + d_a$	$d_s + \mathcal{N} \cdot d_a$
Size of Hidden Layers	256, 256	256, 256
Size of Output Layer	d_a	1
Activation Function for Inputs	-	-
Activation Function for Hidden Layers	ReLU	ReLU
Activation Function for Outputs	Tanh	-

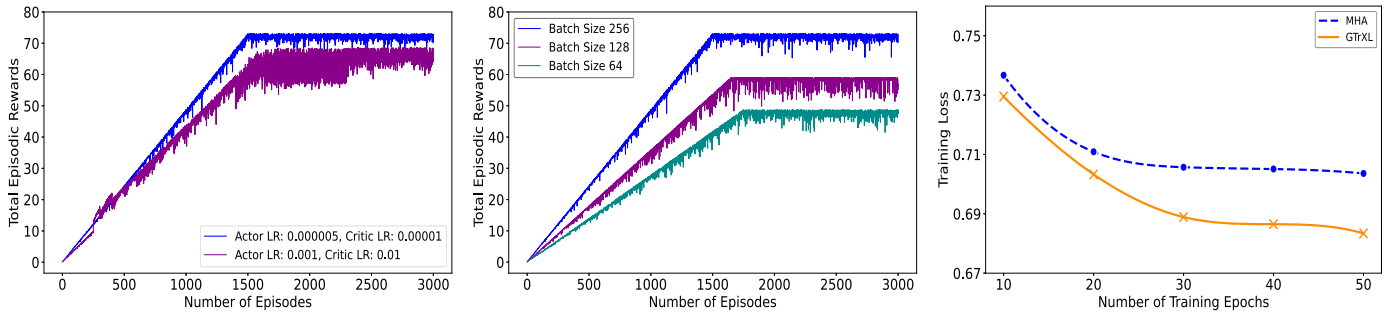
over 30 independent simulation runs for statistical robustness. Table 3 summarizes the major simulation parameters and their ranges.

Table 4 illustrates the hyperparameter settings used to train the GLEMATO framework and all baseline frameworks. The training spans 3000 episodes with 100 time steps to ensure comprehensive exploration of the dynamic multi-agent environment. Actor-critic optimization is carried out using Adam optimizers, supported by exploration noise and an appropriate discount factor of future rewards. Training employs mini-batch gradient descent with regularization to mitigate overfitting. Besides, the Table 5 outlines the configuration of different parameters for the actor and critic neural networks employed within the GLEMATO framework. The actor networks include 1 input layer followed by two hidden layers with 256 neurons each, terminating with an output layer of size d_a representing action values. Similarly, the critic network is composed of 1 input layer and 2 hidden layers with the same neuron configuration, concluding with 1 output neuron for estimating state-action values. The input dimension of the actor is defined as $(d_s + d_a)$, where, $d_s = (Num_{AEC} \times Fea_{AEC}) + (Num_{IoT} \times Fea_{IoT})$, denotes the state dimensions based on the number of AEC servers and IoT devices with their features, and the action spaces determined by the number of AEC servers and IoT devices respectively, incorporating features from both the agent and its neighbors, is represented as $d_a = (Num_{AEC} \times Act_{AEC}) + (Num_{IoT} \times Act_{IoT})$. In contrast, the critic network input dimension is expanded to $(d_s + \mathcal{N} \cdot d_a)$, incorporating global action representations across all agents. Both networks employ $ReLU$ activations in their hidden layers to introduce non-linearity and improve convergence. The actor network uses a $Tanh$ activation in its output layer to constrain actions within the desired range, whereas the critic uses no output activation, allowing it to produce unbounded continuous Q-values for precise value estimation.

6.2. Performance metrics

To compare the effectiveness of different task offloading systems, we've used a set of key performance indicators that reflect how well each system handles real-world demands.

- **Average Energy Consumption (kJ)** reflects the total energy used by both the AEC servers and IoT devices during the task execution



(a) Convergence under different learning rate configurations (b) Convergence under different batch configurations (c) Convergence of Training Loss for GTrXL and MHA

Fig. 3. Training performance and convergence behavior of the GLEMATO (GTrXL + MADDPG) framework under varying hyperparameters.

and communication. Lower energy consumption indicates the system remains operational longer.

- **Average Service Latency (ms)** captures the average time it takes for a task to be transmitted from an IoT device → AEC Server, to be fully processed and computed, which is denoted as $T^{tot}_{i,u}(t)$. Shorter delays is crucial in time-sensitive computation scenarios.
- **Average Network Throughput (Mbps)** measures how efficiently the system can transfer data and manage tasks between IoT devices and AEC servers. Higher throughput means the system can handle more tasks at once, resulting in smoother operations and quicker response times.
- **Task Completion Ratio (%)** η indicates the percentage of tasks that are successfully completed within a certain time window. A higher rate represents a more reliable system, defined as,

$$\eta = \frac{\sum_{i=1}^{I}\sum_{u=1}^{U}\left(\eta_{i,u}(t)\times M_i(t)\right)}{\mathcal{M}_i}, \tag{52}$$

where $\mathcal{M}_i = \sum_{i=1}^{I} M_i(t)$ denotes the total size of all the generated IoT tasks. The $\eta_{i,u}(t)$ represents the total fraction of each task successfully processed in the three-stage at time slot t, which can be determined by,

$$\eta_{i,u}(t) = \overbrace{\left(1-\Psi^{n,off}_{i,u}(t)\right)}^{IoT} + \overbrace{\Psi^{n,off}_{i,u}(t)\times\left(1-\Omega^{n,off}_{i,u'\vee u+1}(t)\right)}^{AEC\ server} + \underbrace{\Psi^{n,off}_{i,u}(t)\times\Omega^{n,off}_{i,u'\vee u+1}(t)}_{Neighbor\ AEC\ server\ or\ Airship}. \tag{53}$$

6.3. Training performance and visualization

To evaluate the robustness and generalization ability of the developed GLEMATO framework, we perform a comprehensive sensitivity analysis, as shown in Fig. 3, over 3000 training episodes, focusing on two pivotal hyperparameters: learning rate and mini-batch size within the MADDPG algorithm. As illustrated in Fig. 3(a), a higher configuration (actor learning rate: 0.001, critic learning rate: 0.01) leads to learning instability and significant fluctuations in reward. In contrast, a lower configuration (actor learning rate: 0.000005, critic learning rate: 0.00001) demonstrates more stable learning and consistent convergence behavior. The learning curves depict an impetuous initial rise in reward, with convergence stabilizing near the 1500th episode. This consistent convergence, sustained throughout the full span of 3000 training episodes, underscores the long-term consistency and the durability of the learned policies achieved through the chosen hyperparameter configuration. In Fig. 3(b), we evaluate the model's performance using distinct batch sizes of 64, 128, and 256. A clear positive correlation between batch size and performance is evident, with the batch size of 256 acquiring the maximum episodic reward (~73) and the earliest convergence, while batch sizes of 128 and 64 converge slowly with lower reward values (~57 and ~49, respectively). Larger batches enhance MADDPG training by leveraging experience replay with diverse training samples and a centralized critic, granting more consistent gradient estimates, reduced update variance, improved learning stability, and faster convergence compared to smaller batches. The loss of GTrXL decreases more rapidly and converges lower than standard MHA even under the same MSE loss as Eq. (47), shown in Fig. 3(c). This gain stems from its Gated Residual Units, which stabilize temporal signal flow and prevent attention saturation during training. Additionally, the GTrXL's recurrent memory path reuses historical hidden states to capture long-range dependencies, enabling more accurate and lower-variance priority regression.

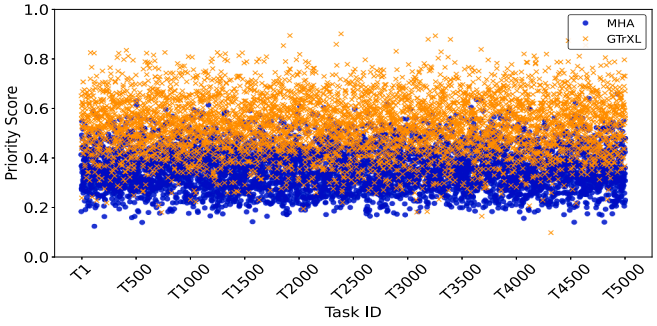
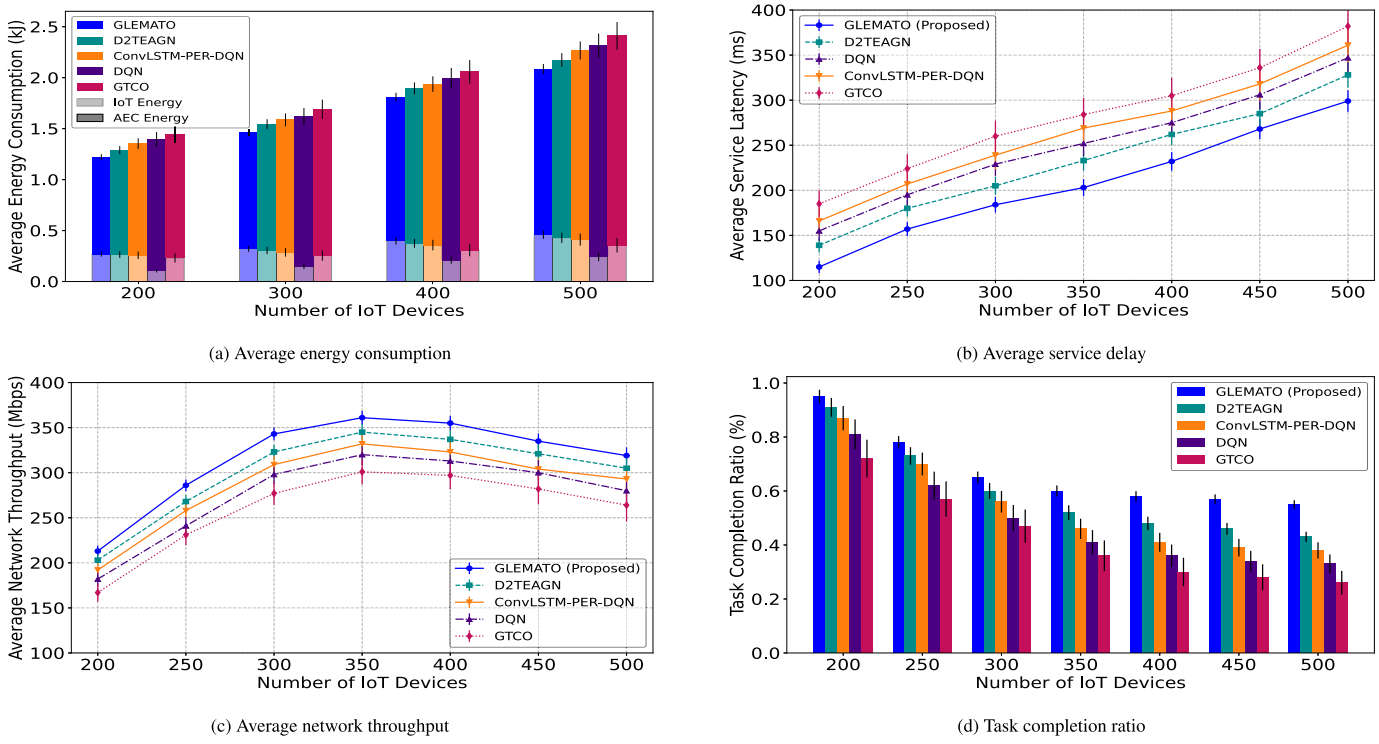


Fig. 4. Comparison of tasks priority score generated by MHA and GTrXL.

Fig. 4, presents an output comparison analysis of deep learning based task prioritization in an AEC-assisted IoT environment, where we investigated two attention-driven architectures, MHA and GTrXL, trained on our synthetic dataset to realistically model spatial distribution and priority dynamics for the GLEMATO framework. While MHA distributes attention uniformly, GTrXL integrates gated residual connections and memory mechanisms with relative positional encoding, enabling selective feature amplification and long-range dependency modeling. This results in a more distinctive separation of high- and low-priority tasks, particularly highlighting delay-sensitive and resource-critical ones. Such sharper prioritization supports improved resource scheduling in AEC server networks, with the generated scores subsequently guiding the MADDPG algorithm to facilitate cooperative policy learning, enhance situational awareness, and contribute to lower energy consumption and improved training efficiency of the MADDPG algorithm.

6.4. Simulation results

In the subsection, we present a rigorous evaluation of the performance of the developed GLEMATO framework by exploring the effects of varying the number of IoT devices, AEC servers, and task execution deadlines.



(a) Average energy consumption

(b) Average service delay

(c) Average network throughput

(d) Task completion ratio

Fig. 5. Impacts of varying number of IoT devices.

6.4.1. Impacts of varying number of IoT devices

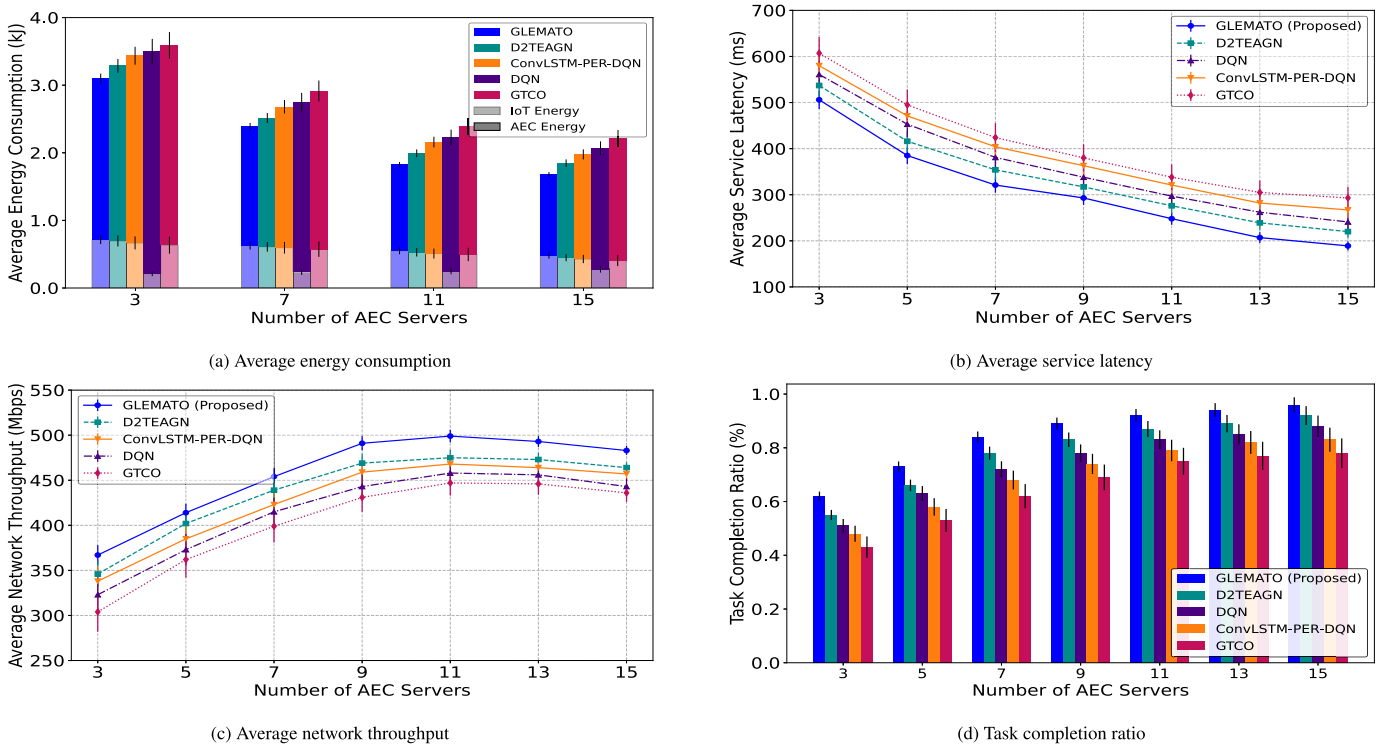
In this experiment, the number of IoT devices varies from 200 to 500 with 10 AEC servers and task deadlines of 500-3500 ms, with devices evenly divided into low, medium, and high-priority categories to reflect realistic heterogeneous workloads.

Fig. 5(a) illustrates the total and layer-wise energy consumption as IoT device density increases, revealing how each framework distributes computational load across the IoT and AEC layers. The GLEMATO shows slightly higher IoT-side energy because its GTrXL-based prioritization strategically retains light weight, delay-tolerant tasks at end the devices, preventing unnecessary offloading. This, combined with MADDPG-driven cooperative scheduling, enables balanced server utilization and avoids cloud escalation, ultimately achieving lower total energy consumption than all baselines. In contrast, the D2TEAGN with air-ground coordination and the ConvLSTM-PER-DQN with short-term workload forecasting achieve moderate IoT-side efficiency but incur additional processing overhead and lack robust predictive load balancing, leading to elevated AEC-side energy. The DQN suffers from fragmented subtask division and uncoordinated offloading, which substantially increases energy costs across the AEC servers. Paradoxically, its minimal local computation, limited solely to task transmission, results in lower IoT-layer energy consumption than the other frameworks. The GTCO performs the worst, exhibiting consistently high energy consumption due to its static, single-agent allocation strategy that overloads servers with excessive offloading.

Fig. 5(b) shows that the average service delay rises for all frameworks with the increasing number of IoT devices. As denser device populations generate more tasks and thereby increase queue waiting times across all traffic levels, however, the proposed GLEMATO framework is consistently able to achieve the lowest delay. The GLEMATO framework is able to preserve substantially shorter queue lengths for time-sensitive tasks, largely due to its GTrXL-driven temporal encoding, which captures recent variations in load and allows the policy to prioritize urgent requests more accurately. Furthermore, GLEMATO's cooperative MADDPG policy promotes early redistribution of tasks among AEC servers, preventing localized congestion and reducing task accumulation at overloaded nodes. The D2TEAGN performs competitively under moderate traffic levels and, due to a lack of U2U or inter-server cooperation, is increasingly prone to queueing delay. The ConvLSTM-PER-DQN, despite its forecasting capability, experiences higher latency than DQN in dense traffic scenarios. This occurs because the recurrent ConvLSTM layer introduces additional sequential update delays under the burst arrivals scenario; this extra computation time slows scheduling responses, allowing queues to accumulate. The DQN, although less energy-efficient, reacts more quickly due to its simpler feedforward neural network structure without having any convolutional LSTM layer and thus maintains comparatively lower delay under heavy load. The remaining GTCO framework remains the weakest performer due to its static task assignment, which cannot adapt to dynamic queue states, leading to persistent congestion.

As shown in Fig. 5(c), network throughput initially increases with the number of IoT devices due to the rise in transmitted tasks, but begins to decline once channel congestion and interference increase. The GLEMATO framework maintains the highest throughput across all device densities by maintaining a lower rate of transmission collisions and reduced retransmission attempts through more stable offloading sequences, balanced server utilization, and higher priority tasks, which will be given priority to transmit tasks using allocated bandwidth. D2TEAGN performs moderately but lacks U2U-aware load redistribution and temporal prioritization, causing sub-channel allocations to drift from actual queue pressures. ConvLSTM-PER-DQN surpasses DQN due to short-term workload prediction, though prediction drift under dense traffic leads to early throughput saturation. DQN offers only moderate throughput because of uneven sub-channel usage, while GTCO performs the worst due to static, single-agent decisions that cannot adapt to evolving interference conditions.

As the number of IoT devices increases, Fig. 5(d) clearly shows a gradual decline in the task completion ratio across all frameworks. This degradation is expected, as more resources are required for task completion due to higher device density intensifies task generation and



(a) Average energy consumption

(b) Average service latency

(c) Average network throughput

(d) Task completion ratio

Fig. 6. Impacts of varying number of AEC servers.

subsequently increases queueing pressure and channel contention. Despite this growing load, the GLEMATO consistently achieves the highest completion ratio at every device count. The taller GLEMATO bars across all clusters show its stronger ability to prevent deadline violations and maintain stable execution under congestion. It achieves this by keeping AEC-server utilization balanced and limiting queue buildup, ensuring timely task processing even near network saturation. D2TEAGN performs reasonably at low densities but falls behind GLEMATO due to missing inter-UAV load coordination. DQN and ConvLSTM-PER-DQN bars drop off more sharply because of weaker prioritization and scheduling stability. GTCO remains the lowest across all densities owing to its static, non-adaptive allocation strategy.

6.4.2. *Impacts of varying number of AEC servers*

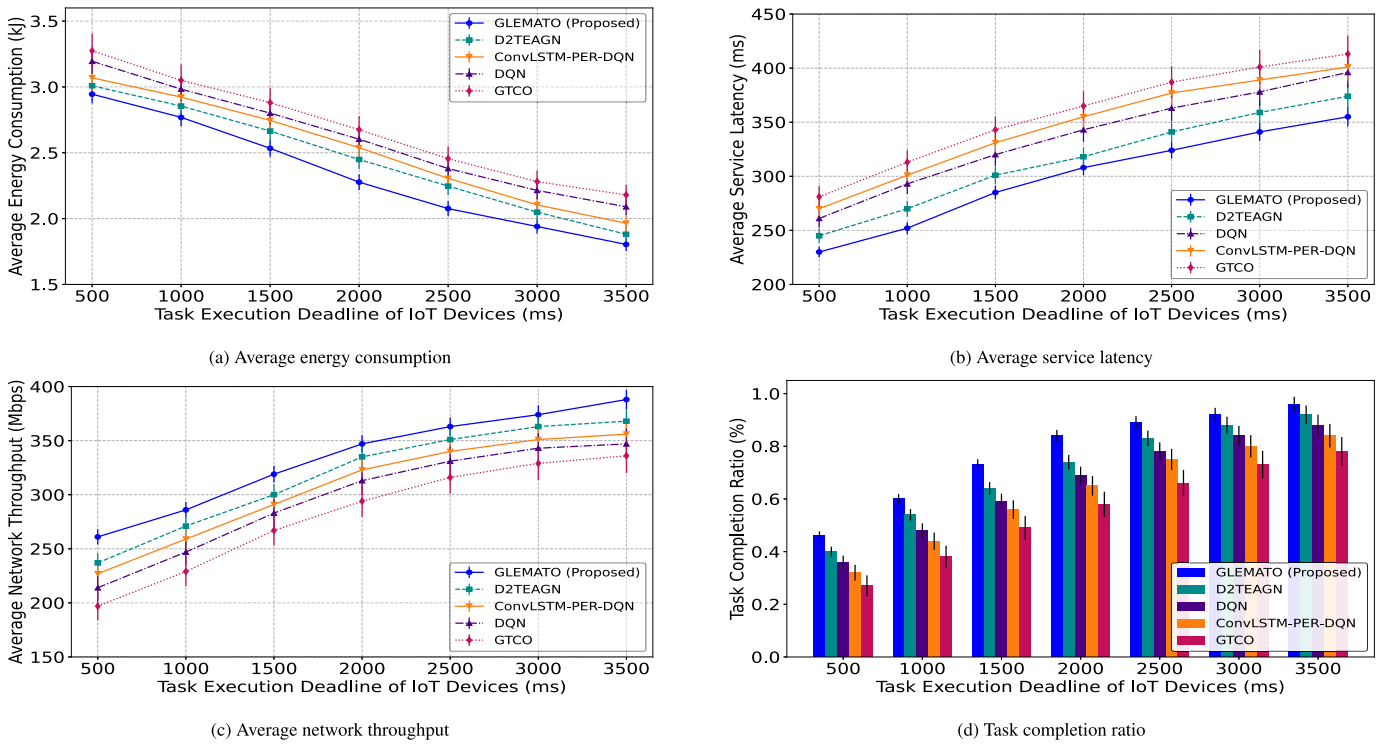
This section examines the performance impact of increasing the number of AEC servers from 3 to 15, while fixing IoT devices at 300 and maintaining the task delay deadline between 500 ms to 3500 ms.

Fig. 6(a) depicts that increasing the number of AEC servers leads to a consistent reduction in total energy consumption across all frameworks, driven by the more effective distribution of workloads across multiple computational AEC servers. The GLEMATO minimizes overall energy consumption by leveraging MADDPG-based cooperative offloading with EWMA-guided load tracking and coordinated task forwarding to sustain evenly balanced, congestion-free, and low-overhead AEC-server utilization. Its GTrXL module suppresses redundant reassignments and wasteful offloading by selectively retaining lightweight subtasks locally, reducing unnecessary uplink traffic despite a slight rise in IoT-side energy. The D2TEAGN system lacks of load balancing at higher AEC densities leads to increased energy consumption. In contrast, ConvLSTM-PER-DQN reduces energy by anticipating workload fluctuations and filtering redundant offloading. However, its recurrent computation slows decision responses and leads to higher latency than DQN. Meanwhile, DQN's excessive subtask splitting and limited local computation trigger frequent offloading at higher AEC densities, increasing IoT-side energy. The GTCO consistently performs the worst because static allocation leads to persistent inefficiencies.

Fig. 6(b) shows that the average service delay decreases as the number of AEC servers increases. With more servers available, tasks experience shorter waiting times in both processing and offloading queues. The GLEMATO consistently records the lowest delay because its GTrXL-enhanced state representation allows it to detect and react promptly to server congestion patterns. This leads to early redistribution of incoming tasks and a more balanced utilization landscape, preventing server-level bottlenecks. The D2TEAGN approaches perform well at moderate server counts, as increased server availability reduces the need for elaborate intra-network cooperation. However, when server density becomes higher, the lack of explicit task prioritization in D2TEAGN makes it unable to fully utilize the expanded capacity like the proposed one. The DQN provides lower latency than ConvLSTM-PER-DQN due to its simpler feedforward neural network structure without having any convolutional LSTM layer and thus achieves comparatively lower delay under heavy load. While GTCO benefits least, as its static strategy does not adapt to changing server availability.

Fig. 6(c) indicates that throughput increases with the number of AEC servers and plateaus when server demand exceeds traffic demand from IoT devices. The GLEMATO framework achieves the highest throughput by maintaining stable offloading patterns, achieving lower collision rates, and reducing retransmissions on both U2IoT and U2U links. Although the D2TEAGN benefits from optimized trajectory and association decisions, its inability to coordinate load across multiple servers limits its gains at higher densities. The ConvLSTM-PER-DQN outperforms standard DQN in average throughput utilization due to its short-term workload prediction capability, whereas DQN's performance is limited by uneven sub-channel utilization. Finally, GTCO exhibits minimal improvement due to its static allocation does not exploit the additional server capacity.

When the number of AEC servers increases, the bar groups in Fig. 6(d) show a steady rise in task completion ratio for every



(a) Average energy consumption

(b) Average service latency

(c) Average network throughput

(d) Task completion ratio

Fig. 7. Impacts of varying task execution deadlines.

framework. This behavior arises from greater computational parallelism and reduced queueing intensity as more servers become available. The proposed framework GLEMATO consistently exhibits the tallest bar in each server configuration, supported by its cooperative coordination and priority-aware task distribution, which efficiently balances load and preserves deadline satisfaction. D2TEAGN also improves but the gap between its bars and GLEMATO's remains visible because it lacks explicit inter-server redistribution. The DQN and ConvLSTM-PER-DQN benefit from the additional servers but still underperform compared to GLEMATO due to weaker prioritization and less stable load balancing, while GTCO maintains the lowest completion ratio due to its static operational behavior.

6.4.3. *Impacts of varying task execution deadlines*

In these experiments, we vary the deadline for executing tasks on IoT devices by considering a fixed number of AEC servers in the simulation at 10, and a total of 300 IoT devices.

In Fig. 7(a), the trend portrays that average energy consumption decreases with more relaxed IoT devices task execution deadlines. This correlates with extended operational durations and reduced scheduling urgency, leading to suboptimal resource utilization and task offloading during expanded execution windows. The GTCO records the highest energy use because a single server handles most tasks without coordination. The D2TEAGN, DQN, and ConvLSTM-PER-DQN perform better with multiple AEC servers but lack strong prioritization and robust mechanisms to maximize adaptive task scheduling and computational load distributions among neighboring nodes under varying task execution deadline constraints. However, the GLEMATO achieves the lowest energy consumption by combining GTrXL-based prioritization with MADDPG-driven cooperative offloading by avoiding unnecessary data transmissions.

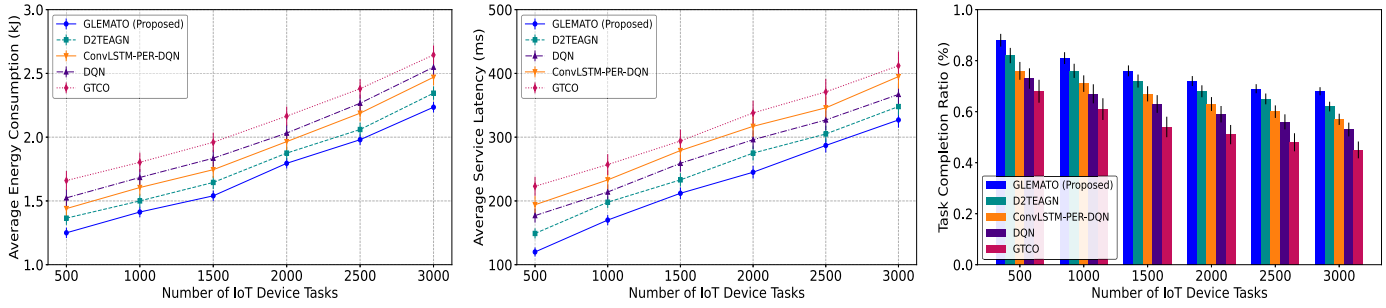
Fig. 7(b) depicts average service delay, which increases with relaxed deadlines as systems prioritize energy saving and load balancing. The GLEMATO maintains the lowest delays through GTrXL-based prioritization of emergent tasks and MADDPG-based cooperative offloading strategy. Additionally, its inter-agent collaboration enhances load balancing and reduces queuing and execution delays, outperforming all baselines. Fig. 7(c) reveals an upward trend in network throughput with increased deadlines. With longer task execution deadlines, AEC servers will process and offload more data, enhancing throughput for the network. The GLEMATO consistently achieves the highest throughput due to its GTrXL-driven prioritization policy, which the early processing of bandwidth-heavy tasks and leverages U2U coordination to reduce bottlenecks as well as enhance transmission flow, enabling the GLEMATO to outperform baseline methods in dynamic AEC-assisted IoT environments.

Fig. 7(d) illustrates that the task completion ratio steadily improves with extended deadlines. The GLEMATO achieves the highest completion rates by integrating prioritization with cooperative scheduling, while the GTCO performs the worst due to its single-agent design. In contrast, D2TEAGN, ConvLSTM-PER-DQN, and DQN incorporate multi-agent setups but lack effective task prioritization and load balancing strategies which hinder their overall performance. This exhibits its advanced capacity in optimizing task offloading and resource utilization to maximize completion rates in critical IoT environments.

6.4.4. *Traffic sensitivity and scalability analysis*

To evaluate traffic sensitivity and scalability, we vary the task load generated by 300 heterogeneous IoT devices within a 4-minute simulation window, while keeping the number of AEC servers fixed at 10. Across this window, the IoT layer produces 500-3000 tasks, corresponding to approximately 2-10 tasks per device, with task-generation intervals spanning 144 seconds under light traffic to 24 seconds at peak load.

Under rising IoT task-generation rates, Figs. 8(a–c) collectively demonstrate that higher traffic intensifies uplink activity, deepens queue interactions, and increases deadline-violation likelihood across all frameworks. The GLEMATO demonstrates superior scalability due its integrated task-prioritization and cooperative offloading strategy minimizes redundant transmissions, aligns tasks with real-time server capacity, and preserves stable processing flows even under bursty arrivals.



(a) Average energy consumption (b) Average service latency (c) Task Completion Ratio

Fig. 8. Impacts of varying number of IoT devices tasks.

Table 6
Computational load balance (Std. Dev.) across AEC servers.

AEC Servers	GLEMATO *(Proposed)*	D2TEAGN	DQN	ConvLSTM-PER-DQN	GTCO
3	**0.52**	0.58	0.61	0.67	0.72
7	**0.34**	0.42	0.49	0.58	0.65
11	**0.28**	0.33	0.38	0.48	0.60
15	**0.21**	0.26	0.29	0.36	0.58

Table 7
Task offloading ratio of GLEMATO across AEC servers.

AEC Servers	Local Computing (IoT)	Offload to Airship(Cloud)	Offload to AEC Server
3	0.20	0.23	0.57
7	0.17	0.16	0.66
11	0.13	0.13	0.74
15	0.10	0.11	0.79

The D2TEAGN benefits from trajectory-efficient UAV access and offers moderate robustness, but its absence of cross-server workload coordination limits its effectiveness as arrival densities escalate. Conversely, ConvLSTM-PER-DQN and DQN respond reactively, causing oscillatory offloading, uneven queue buildup, and congestion driven degradation. The GTCO performs the worst, as its static routing consistently overloads specific servers, resulting in excessive retransmissions, prolonged delays, and significant task drops under heavy traffic.

6.4.5. *Load balancing and task offloading ratio under servers scaling*

The Table 6 illustrates the effectiveness of the computational load balancing model is empirically evaluated using $\sigma_u^{com}(t)$, among AEC servers measured by Eq. (19). As the number of servers increases, while fixing the number of IoTs at 300, the overall load of the system decreases from the server perspective, indicating that a lower value of $\sigma_u^{com}(t)$ is more effective and providing fair load distribution among the servers. The GLEMATO framework consistently achieves the lowest imbalance rate due to its scalable task distribution, compared to baseline schemes that lack explicit coordination-aware load regulation and dynamic redistribution capability, leading to higher load imbalance scenarios.

Table 7 shows, the task offloading ratio in the GLEMATO framework progressively shifts computation from IoT devices and the Airship toward the AEC servers. This shift is driven by the GTrXL-based temporal prioritization module, which offloads urgent and computation-heavy tasks to the expanded AEC capacity. The MADDPG cooperative policy further balances workloads across neighboring AEC servers. Together, these mechanisms increase server-side execution while reducing energy-intensive local and cloud processing, enabling a scalable and resource-aware task offloading strategy.

The overall results demonstrate that GLEMATO excels in balancing delay, energy efficiency, optimal task offloading, and load balancing across diverse conditions. While other baselines perform reasonably in specific scenarios, none match GLEMATO's ability to manage tasks and resources holistically. This makes it a robust and reliable choice for large-scale, dynamic AEC-assisted IoT environments, ensuring efficient and high-quality service even under complex workloads.

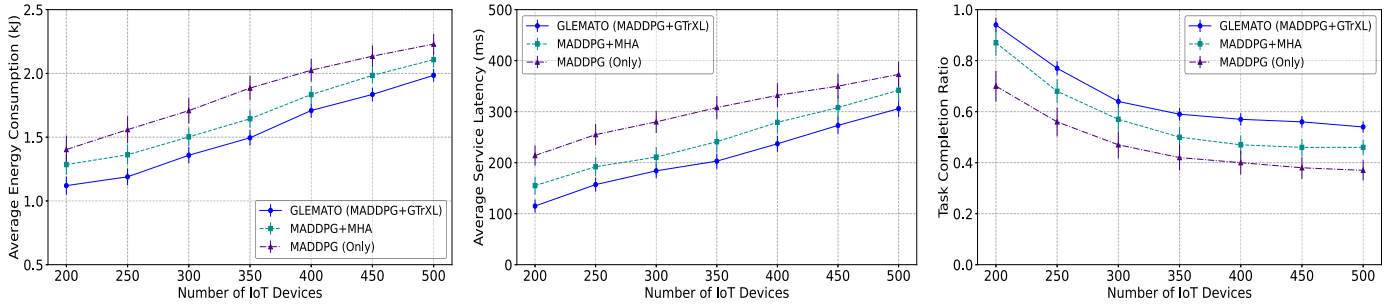
6.4.6. *Ablation experiments*

To isolate the contribution of integrating the GTrXL-based priority module into the MADDPG, we perform an ablation study comparing three variants: GLEMATO (MADDPG + GTrXL), MADDPG + MHA, and the baseline MADDPG (Only). The comparison is performed under a fixed AEC server count of 10 while varying the number of IoT devices, enabling a clear assessment of each module's impact on scalability, coordination, and performance in AEC-assisted IoT environments.

As shown in Fig. 9(a), the GLEMATO consistently achieves the lowest average energy consumption, with the gap increasing as number of IoT devices increases. This improvement stems from GTrXL's gated memory, which enables more accurate prioritization and and coordination of task offloading. MADDPG + MHA performs better than the baseline through improved short-term attention but loses efficiency under heavier workloads due to limited temporal stability. Fig. 9(b) illustrates the service delay across GLEMATO, MADDPG + MHA, and baseline MADDPG as the IoT load increases. The GLEMATO consistently achieves the lowest latency, driven by GTrXL's gated temporal memory that stabilizes long-range dependencies and improves task scheduling under congestion. Finally, Fig. 9(c) shows that GLEMATO achieves the highest task completion ratio, due to GTrXL's ability to preserve priority consistency and ensure reliable task execution with better resource-task alignment. In contrast, MADDPG + MHA offers moderate gains, while the baseline demonstrates the lowest reliability. In general, the integration of GTrXL and MADDPG within GLEMATO results in notable improvements in energy usage, latency, and task completion ratio, highlighting the effectiveness of GTrXL-based multi-agent learning for scalable AEC environments.

6.4.7. *Overhead evaluation*

Overhead evaluations refer to the total signaling traffic exchanged among IoT devices, AEC servers, and the DT that supports coordination, training, and real-time decision-making. Only non-payload control messages, such as DT synchronization updates, inter-agent state-action vectors used during centralized MADDPG training, and GTrXL-generated priority scores are considered in this calculation, while task data traffic is explicitly excluded. Control overhead is quantified by measuring the



(a) Average energy consumption (b) Average service latency (c) Task completion ratio

Fig. 9. Performance impact of integrating the gated transformer-XL model into MADDPG (GLEMATO).

Table 8
Control overhead comparison of baseline schemes and GLEMATO.

Scheme	Control Packet Volume (KB/s)	Coordination Time Ratio (%)	DT Messages	Subtask Control
GTCO	1.05	2.5	✗	✗
DQN	1.65	3.9	✗	✗
ConvLSTM	2.85	6.1	✗	✗
D2TEAGN	1.84	4.2	✓	✗
GLEMATO	**1.96**	**4.6**	✓	✗

size and frequency of these exchanged vectors across all communication links. In addition, we evaluate the Coordination Time Ratio (CTR), defined as the proportion of total wireless airtime consumed by control signaling relative to overall channel usage. Together, these metrics capture both the bandwidth cost and channel occupancy introduced by cooperative decision-making in AEC environments.

Table 8 compares the control overhead characteristics of GLEMATO with key baseline schemes. GTCO and DQN exhibit relatively low control packet volumes and coordination time ratios but lack DT support and multi-UAV cooperation, limiting their scalability. ConvLSTM-PER-DQN incurs higher overhead due to recurrent feature exchanges required for predictive state modeling. D2TEAGN benefits from DT integration but relies on frequent periodic synchronization without event-triggering, which increases signaling load as UAV density grows. In contrast, GLEMATO introduces hybrid (periodic + event-driven) DT updates and transmits compact GTrXL-generated priority vectors, resulting in moderate control packet usage and stable coordination time even under large-scale deployments.

7. Conclusion and future work

In this paper, we developed a three-stage DT-enabled optimization framework that exploits MINLP to jointly minimize service latency and energy consumption by offloading computationally intensive IoT tasks in an AEC-assisted IoT environment. To address the intrinsic complexity of the NP-hard problem, we introduced a practically scalable framework namely, GLEMATO, a GTrXL-driven MADDPG framework that provides fine-grained task prioritization and cooperative multi-agent decision-making. By leveraging AEC servers and cloud collaboration, DT-enabled synchronization, GTrXL's gating mechanisms, and MADDPG's adaptive policy learning, the GLEMATO adapts effectively to varying IoT traffic and AEC server densities. The experimental results proved that the GLEMATO outperformed state-of-the-art methods, achieving an average reduction in energy consumption of 21.8% and in service latency of 23.3%, while increasing the average task completion ratio by around 20.1% and the average throughput by 17.1%.

In the future, the performances of the GLEMATO framework can be extended through modeling security-aware and privacy-preserving task executions exploiting asynchronous federated learning, blockchain-based resource trading, and AI-driven resilience. Furthermore, incorporating green AI principles could improve sustainability in real-world 6G and IoT-enabled edge deployments.

CRediT authorship contribution statement

Md. Abdullah Al Sami: Writing – original draft, Methodology, Investigation, Data curation, Conceptualization; **Ibrahim Tanvir:** Writing – original draft, Software, Methodology, Investigation, Conceptualization; **Palash Roy:** Writing – review & editing, Visualization, Supervision, Methodology, Investigation; **Md. Abdur Razzaque:** Writing – review & editing, Visualization, Supervision, Project administration, Methodology; **Md Rafiul Hassan:** Writing – review & editing, Validation, Resources; **Mohammad Mehedi Hassan:** Writing – review & editing, Funding acquisition, Formal analysis.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors are also grateful to King Saud University, Riyadh, Saudi Arabia, for funding this work through the Ongoing Research Funding Program (ORF-2026-18).

References

[1] B. Chen, H. Zhou, J. Yao, H. Guan, RESERVE: An energy-efficient edge cloud architecture for intelligent multi-UAV, IEEE Trans. Serv. Comput. 15 (2) (2019) 819–832.
[2] Y.M. Park, S.S. Hassan, Y.K. Tun, Z. Han, C.S. Hong, Joint trajectory and resource optimization of MEC-assisted UAVs in sub-THz networks: a resources-based multi-agent proximal policy optimization DRL with attention mechanism, IEEE Trans. Veh. Technol. 73 (2) (2024) 2003–2016.
[3] Precedence Research, Unmanned Aerial Vehicle, Drones Market Size, Share, and Trends, 2024, [Online; accessed 12-September-2024].
[4] H.-T. Ye, X. Kang, J. Joung, Y.-C. Liang, Optimization for wireless-powered IoT networks enabled by an energy-limited UAV under practical energy consumption model, IEEE Wireless Commun. Lett. 10 (3) (2021) 567–571.

[5] S. Brotee, F. Kabir, M.A. Razzaque, P. Roy, M. Mamun-Or-Rashid, M.R. Hassan, M.M. Hassan, Optimizing UAV-UGV coalition operations: a hybrid clustering and multi-agent reinforcement learning approach for path planning in obstructed environment, Ad Hoc Netw. 160 (2024) 103519.
[6] M. Hevesli, A.M. Seid, A. Erbad, M. Abdallah, Task offloading optimization in digital twin assisted MEC-Enabled air-ground IIoT 6G networks, IEEE Trans. Veh. Technol. 73 (11) (2024) 1–16.
[7] X. Li, J. Wei, H. Wang, L. Dong, R. Chen, C. Yi, J. Cai, D. Niyato, X. Shen, Towards intelligent tansportation with pedestrians and vehicles in-the-loop: a surveillance video-assisted federated digital twin framework, IEEE Netw. 39 (6) (2025) 305–315.
[8] S.D. Okegbile, J. Cai, H. Zheng, J. Chen, C. Yi, Differentially private federated multi-task learning framework for enhancing human-to-virtual connectivity in human digital twin, IEEE J. Sel. Areas Commun. 41 (11) (2023) 3533–3547.
[9] X. Tang, X. Li, R. Yu, Y. Wu, J. Ye, F. Tang, Q. Chen, Digital-twin-assisted task assignment in multi-UAV systems: a deep reinforcement learning approach, IEEE Internet Things J. 10 (17) (2023) 15362–15375.
[10] M.A. Al Sami, I. Tanvir, M.A.T. Siddique, P. Roy, M.A. Razzaque, M.A. Rahaman, Energy and Latency-Aware Optimal Task Allocation in Edge Computing Assisted Multi-UAV System, in: 2024 6th International Conference on Sustainable Technologies for Industry 5.0 (STI), Rupganj, Narayanganj, Bangladesh, 2024, pp. 1–6.
[11] F. Zhou, R.Q. Hu, Z. Li, Y. Wang, Mobile edge computing in unmanned aerial vehicle networks, IEEE Wireless Commun. 27 (1) (2020) 140–146.
[12] M.-A. Messous, S.-M. Senouci, H. Sedjelmaci, S. Cherkaoui, A game theory based efficient computation offloading in an UAV network, IEEE Trans. Veh. Technol. 68 (5) (2019) 4964–4974.
[13] K. Zhang, X. Gui, D. Ren, D. Li, Energy–latency tradeoff for computation offloading in UAV-assisted multiaccess edge computing system, IEEE Internet Things J. 8 (8) (2020) 6709–6719.
[14] H. Yuan, M. Wang, J. Bi, S. Shi, J. Yang, J. Zhang, M. Zhou, R. Buyya, Cost-efficient task offloading in mobile edge computing with layered unmanned aerial vehicles, IEEE Internet Things J. 11 (19) (2024) 30496–30509.
[15] Z. Wang, W. Zhao, P. Hu, X. Zhang, L. Liu, C. Fang, Y. Sun, UAV-assisted mobile edge computing: dynamic trajectory design and resource allocation, Sensors 24 (12) (2024).
[16] G. Li, J. Cai, An online incentive mechanism for collaborative task offloading in mobile edge computing, IEEE Trans. Wireless Commun. 19 (1) (2020) 624–636.
[17] H. Cao, J. Cai, Distributed multiuser computation offloading for cloudlet-based mobile cloud computing: a game-theoretic machine learning approach, IEEE Trans. Veh. Technol. 67 (1) (2018) 752–764.
[18] M. Wu, Y. Xiao, Y. Gao, M. Xiao, Digital twin for UAV-RIS assisted vehicular communication systems, IEEE Trans. Wireless Commun. 23 (7) (2024) 7638–7651.
[19] H. Guo, X. Zhou, J. Wang, J. Liu, A. Benslimane, Intelligent task offloading and resource allocation in digital twin based Aerial computing networks, IEEE J. Sel. Areas Commun. 41 (10) (2023) 3095–3110.
[20] B. Hazarika, K. Singh, C.-P. Li, A. Schmeink, K.F. Tsang, RADiT: resource allocation in digital twin-Driven UAV-aided internet of vehicle networks, IEEE J. Sel. Areas Commun. 41 (11) (2023) 3369–3385.
[21] B. Hazarika, K. Singh, A. Paul, T.Q. Duong, Hybrid machine learning approach for resource allocation of digital twin in UAV-aided internet-of-vehicles networks, IEEE Trans. Intell. 9 (1) (2024) 2923–2939.
[22] P. Consul, I. Budhiraja, D. Garg, N. Kumar, R. Singh, A.S. Almogren, A hybrid task offloading and resource allocation approach for digital twin-empowered UAV-assisted MEC network using federated reinforcement learning for future wireless network, IEEE Trans. Consum. Electron. 70 (1) (2024) 3120–3130.
[23] X. Huang, Y. Zhang, Y. Qi, C. Huang, M.S. Hossain, Energy-efficient UAV scheduling and probabilistic task offloading for digital twin-empowered consumer electronics industry, IEEE Trans. Consum. Electron. 70 (1) (2024) 2145–2154.
[24] Y. Wang, C. Zhang, T. Ge, M. Pan, Computation offloading via multi-agent deep reinforcement learning in aerial hierarchical edge computing systems, IEEE Trans. Network Sci. Eng. 11 (6) (2024) 5253–5266.
[25] M. Betalo, S. Leng, N. Abishu, F. Dharejo, M. Seid, A. Erbad, R. Naqvi, L. Zhou, M. Guizani, Multi-agent deep reinforcement learning-based task scheduling and resource sharing for O-RAN-empowered multi-UAV-assisted wireless sensor networks, IEEE Trans. Veh. Technol. PP (2023) 1–14.
[26] J. Du, Z. Kong, A. Sun, J. Kang, D. Niyato, X. Chu, F.R. Yu, MADDPG-based joint service placement and task offloading in MEC empowered air-ground integrated networks, IEEE Internet Things J. 11 (6) (2024) 10600–10615.
[27] P. Qin, Y. Wang, Z. Cai, J. Liu, J. Li, X. Zhao, MADRL-based URLLC-aware task offloading for air-ground vehicular cooperative computing network, IEEE Trans. Intell. Transp. Syst. 25 (7) (2024) 6716–6729.
[28] M. Yan, L. Zhang, W. Jiang, C.A. Chan, A.F. Gygax, A. Nirmalathas, Energy consumption modeling and optimization of UAV-assisted MEC networks using deep reinforcement learning, IEEE Sens. J. 24 (8) (2024) 13629–13639.
[29] Y. Li, W. Wang, H. Gao, Y. Wu, M. Su, J. Wang, Y. Liu, Air-to-ground 3D channel modeling for UAV based on gauss-Markov mobile model, AEU Int. J. Electron. Commun. 114 (2020) 152995.
[30] S. Murshed, A.S. Nibir, M.A. Razzaque, P. Roy, A.Z. Elhendi, M.R. Hassan, M.M. Hassan, Weighted fair energy transfer in a UAV network: a multi-agent deep reinforcement learning approach, Energy 292 (2024) 130527.
[31] Q. Wu, R. Zhang, Common throughput maximization in UAV-enabled OFDMA systems with delay consideration, IEEE Trans. Commun. 66 (12) (2018) 6614–6627.
[32] A. Al-Hourani, S. Kandeepan, A. Jamalipour, Modeling air-to-ground path loss for low altitude platforms in urban environments, in: 2014 IEEE Global Communications Conference, Austin, TX, USA, 2014, pp. 2898–2904.
[33] B. Shi, F.-C. Zheng, C. She, J. Luo, A.G. Burr, Risk-resistant resource allocation for eMBB and URLLC coexistence under M/G/1 queueing model, IEEE Trans. Veh. Technol. 71 (6) (2022) 6279–6290.
[34] N.M. Laboni, S.J. Safa, S. Sharmin, M.A. Razzaque, M.M. Rahman, M.M. Hassan, A hyper heuristic algorithm for efficient resource allocation in 5G mobile edge clouds, IEEE Trans. Mob. Comput. 23 (1) (2024) 29–41.
[35] W. Sun, P. Wang, N. Xu, G. Wang, Y. Zhang, Dynamic digital twin and distributed incentives for resource allocation in aerial-assisted internet of vehicles, IEEE Internet Things J. 9 (8) (2022) 5839–5852.
[36] C.W. Chan, T.Y. Kam, A procedure for power consumption estimation of multi-rotor unmanned aerial vehicle, in: The 10th Asian-Pacific Conference on Aerospace Technology and Science, Journal of Physics: Conference Series, 1509, IOP Publishing, China, 2020, p. 012015.
[37] M.R. Garey, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman and Company, New York, USA, 1 edition, New York, USA, 2002.
[38] H.U. Sheikh, L. Bölöni, Multi-agent reinforcement learning for problems with combined individual and team reward, in: Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN), IEEE, Glasgow, United Kingdom, 2020, pp. 1–8.
[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems (NeurIPS 2017), 30, Curran Associates, Inc., Red Hook, NY, USA, 2017, pp. 5998–6008.
[40] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. Le, R. Salakhutdinov, Transformer-XL: attentive language models beyond a fixed-length context, in: A. Korhonen, D. Traum, L. Màrquez (Eds.), Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 2978–2988.
[41] H. Wang, C.H. Liu, H. Yang, G. Wang, K.K. Leung, Ensuring threshold AoI for UAV-assisted mobile crowdsensing by multi-agent deep reinforcement learning with transformer, IEEE/ACM Trans. Netw. 32 (1) (2024) 566–581.
[42] Z. Fu, Y. Mao, D. He, J. Yu, G. Xie, Secure multi-UAV collaborative task allocation, IEEE Access 7 (2019) 35579–35587.
[43] A. Hentati, L. Fourati, E. Elgharbi, S. Tayeb, Simulation tools, environments and frameworks for UAVs and multi-UAV-based systems performance analysis (version 2.0), Int. J. Model. Simul. 43 (2022) 1–17.
[44] N.N. Ei, M. Alsenwi, Y.K. Tun, Z. Han, C.S. Hong, Energy-efficient resource allocation in multi-UAV-assisted two-stage edge computing for beyond 5G networks, IEEE Trans. Intell. Transp. Syst. 23 (9) (2022) 16421–16432.